

**Projet Electronique appliquée
2025-2026 :
Réalisation d'une station météo**

2025 - 2026
BA3 - MA0 Ingénieur Industriel
Orientation Informatique

Table des matières

1	Introduction.....	3
1.1	Cahier des charges	4
1.1.1	Objectifs	4
1.1.2	Liste des composants électroniques essentiels	4
1.1.3	Fonctionnalités attendues.....	4
2	Travail de recherche	4
2.1	Qu'est-ce qu'une station météo ?	4
2.2	Fonctionnement global	5
3	Réalisation du projet	6
3.1	Equipes et travail attendu	6
3.2	Logiciels utilisés.....	6
3.3	Coordination (Equipe1)	7
3.3.1	Déroulement de la conception du projet.....	7
3.3.2	Architecture Logicielle.....	7
3.4	RTC (Equipe2)	7
3.4.1	Présentation du composant	8
3.4.2	Présentation du module RTC	8
3.4.3	Code RTC	9
3.4.4	Visualisation à l'oscilloscope	11
3.5	Menu LCD (Equipe2).....	13
3.5.1	Présentation du composant	13
3.5.2	Code Menu	14
3.6	EEPROM et DATALOGGER (Equipe5)	17
3.6.1	Présentation du composant EEPROM.....	17
3.6.2	Le Protocole de communication	18
3.6.3	Compression des mesures	18
3.6.4	Formatage de la mémoire	19
3.6.5	Code.....	20
3.6.6	Visualisation à l'oscilloscope	21
3.7	Température ambiante et Humidité Relative (Equipe4)	21
3.7.1	Présentation du capteur SHT30	22
3.7.2	Protocole I ² C.....	25
3.7.3	Configuration initiale du capteur	26
3.7.4	Reste du code	27
3.7.5	Conversion des données	29
3.7.6	Oscilloscope.....	30
3.8	Capteur BMP280 (Equipe3).....	32
3.8.1	Présentation du BMP280	32
3.8.2	Initialisation	32

3.8.3	Reste du code	33
3.8.4	Analyse à l'oscilloscope	34
3.9	Equipe application (Equipe6)	35
3.9.1	Présentation du Bluetooth HC-05	36
3.9.2	Caractéristiques techniques	36
3.9.3	Interface UART	36
3.9.4	Principe de fonctionnement.....	37
3.9.5	Rôle dans le projet.....	37
3.9.6	Principe de la communication Bluetooth.....	38
3.9.7	Protocole applicatif	38
3.9.8	Liaison série UART	39
3.9.9	Oscillogrammes	39
3.9.10	Programmation de l'app	41
3.9.11	Architecture Logicielle.....	42
3.9.12	Gestion de la connexion Bluetooth.....	43
3.9.13	Envoi des commandes.....	44
3.9.14	Traitement des données reçues.....	44
3.9.15	Processus de synchronisation	45
3.9.16	Persistance des données.....	45
4	Analyse du produit final	46
5	Conclusion	47
6	Bibliographie	47
7	Annexe.....	47

Image 1 – RTC DS1307.....	9
Image 2 - Ecran LCD.....	15
Image 3 – Contexte du Menu.....	16
Image 4 – Fonctions des champs numériques.....	18
Image 5 - EEPROM.....	18
Image 6 - Schéma du protocole SPI.....	19
Image 7 - Communication SPI : Trame à l'oscilloscope.....	22
Image 8 - Capteur SHT30.....	23
Image 9 - Tolérance de la mesure d'humidité relative à 25 °C pour le capteur SHT30.	25
Image 10 - Tolérance typique de la mesure d'humidité relative selon la température pour le capteur SHT30.	26
Image 11 - Organigramme de la fonction sht30_read()	29
Image 12 - Communication I ² C : Trames à l'oscilloscope.....	32
Image 13 - Capteur BMP280	33
Image 14 - Capteur BMP280	33
Image 15 - Formule de compensation de la température de la documentation du BMP280	36
Image 16 - Formule de compensation de la pression de la documentation du BMP280	36
Image 17 - Octet reçu par le BMP280 capturé avec l'oscilloscope.....	37
Image 18 - Communication UART : envoi d'une commande	42
Image 19 - Communication UART : Réception d'une commande.....	43
 Tableau 1 Répartition des groupes	 7
Tableau 2 - Représentation du Datalogger	21
Tableau 3 - Tableau de configuration du capteur BMP280	34
Tableau 4 - Paramètre de communication pour l'UART	38
Tableau 5 – Câblage du module HC-05	39
Tableau 6 – Liste des commandes disponibles	40
Tableau 7 - Technologies proposées par React Native utilisées dans le projet.....	44

1 Introduction

Le cours d'électronique appliquée nous a introduit au monde de la programmation des microcontrôleurs et ce projet consiste en la conclusion d'un quadrimestre d'apprentissage et de ce fait l'application de nos nouveaux savoirs et savoir-faire.

1.1 Cahier des charges

1.1.1 Objectifs

L'objectif principal est de programmer par modules une station météo miniature capable d'enregistrer intelligemment des mesures à intervalles réguliers et à paramètres variables pouvant être redéfinis entre différentes prises de mesures.

Le deuxième objectif est l'envoi et le traitement de la table des données par le module Bluetooth à un appareil mobile connecté supportant une application Android programmée préalablement pour lire les mesures enregistrées sur un échantillonnage.

1.1.2 Liste des composants électroniques essentiels

La PCB (*printed circuit board*) fournie présentait les composants suivants reliés au microcontrôleur par un protocole chaque :

- Le capteur de température et de pression BMP280 par protocole I²C
- Le capteur de température et d'humidité relative SHT30 par I²C
- La mémoire solide M93C66(EEPROM) par SPI
- L'horloge RTC DS1307 (registres accédés via I²C)
- Le module Bluetooth HC05 communiquant recevant et relayant en UART
- Le microcontrôleur PIC18F26K83
- Un écran LCD pour l'affichage du menu

1.1.3 Fonctionnalités attendues

Outre le bon fonctionnement des capteurs, celles des bus et comment leurs communications sont utilisées par le microcontrôleur, deux fonctionnalités importantes représentent l'interface utilisateur:

- Le menu sur l'écran LCD, servant à commencer la prise de mesure ou d'en changer les paramètres initiaux
- L'application Android, faisant office de format final des mesures et qui peut donc servir de nouveau point de départ à un projet futur (graphiques, prévisions météo etc.)

2 Travail de recherche

2.1 Qu'est-ce qu'une station météo ?

Les stations météorologiques sont un pilier de l'étude du climat et de la surveillance des forces de la nature évoluant constamment au cours du temps. Dispersées à travers le monde elles nous fournissent en temps réels à plusieurs points stratégiques des informations clefs locales comme la température, la pression, l'humidité, l'intensité UV ou la vitesse/la direction du vent. Ces informations sont de première nécessité dans la prévision de phénomènes météorologiques pouvant

affecter de manière néfaste la population du monde entier, que ce soit directement leur mobilité ou leurs habitations, ou plus indirectement par l'agriculture et donc nos récoltes.

Ainsi ces installations sont devenues des outils indispensables au cours du dernier siècle et selon leur emplacement leur bon fonctionnement est primordial pour des parts de populations fort importantes.

2.2 *Fonctionnement global*

Puisque l'humanité ne peut pas agir sur le climat (dans le sens de la régulation du climat à l'échelle planétaire, à part pour la capacité à augmenter sa température), les stations météo ne sont pas des capteurs servant à fermer une boucle de régulation, et les données d'une seule station météo ne peuvent pas nous fournir des prévisions intéressantes concernant le lieu opposé sur le globe terrestre. Ainsi nous utilisons plutôt les stations météo et leurs données comme les pièces dans le grand puzzle du climat global. En mesurer simultanément des données de la même nature dans chaque secteur et en regroupant celles-ci pour être analysées en temps réels ou stockées pour révision future nous pouvons tracker des phénomènes naturels se déplaçant à travers ces secteurs plutôt que de travailler avec des absolus et donc prévoir les destinations de ces derniers et ainsi prévoir plutôt que de réagir.

Ainsi les facteurs déterminants pour les capteurs présents dans ces stations sont leur précision mais surtout leur rapidité ou plus exactement leur réactivité.

3 Réalisation du projet

3.1 Equipes et travail attendu

EQUIPE	MEMBRES
1.COORDINATION	Antoine DUSART – Victor HACHARD
2.LCD + RTC	Selena D'URBANO– Noé VAN SPITAEI
3.BMP280	Simon BOUGARD – Matthias DELCROIX Nicolas QUENON
4.SHT30	Fayssal BOUMADKAR – Neama HMAMI Augustin MICHIELS
5.MÉMOIRE EEPROM + DATALOGGER	Alexis VIVALDI – Rayan MAKKI Alexandre SZCZEPANSKI
6.BLUETOOTH + APP ANDROID	Mathis BURY – Nathan DE SPRINGER

Tableau 1 Répartition des groupes

EQUIPE 1. Les coordinateurs ont dû assurer la bonne avancée de chacun des autres groupes, fournir les détails des fonctions qu'ils devaient programmer et leur rôle dans le projet global, en plus d'assurer la bonne communication entre les groupes pour éviter les conflits de fonction.

EQUIPE 2. L'équipe LCD+RTC a dû s'occuper du menu sur l'écran LCD du PCB et de la boucle principale du programme finale avec les exceptions permettant d'aller chercher les mesures sans intégrer de délai dans les mesures.

EQUIPE 3. L'équipe du capteur BMP280 a mis en place les fonctions permettant d'accéder aux mesures de la température et de la pression en se renseignant sur le format des registres du composants.

EQUIPE 4. Le second capteur a accompli la même manœuvre pour la température et l'humidité relative, bien que leur mesure de la température n'ait pas été exploité car moins précise que celle fournie par BMP280.

EQUIPE 5. L'équipe en charge de stocker les données a dû, en plus de préparer l'écriture et la lecture des données enregistrées de façon structurée, prévoir le formatage des données avec des fonctions de conversion et conversion inverse visant à réduire l'espace mémoire utilisé au coût parfois d'une légère perte de précision.

EQUIPE 6. L'équipe s'occupant d'exploiter les données finales a dû utiliser le module Bluetooth et préparer une application servant à exploiter les données d'une série de prises de mesures pour télécharger l'historique de celle-ci.

3.2 Logiciels utilisés

Pour effectuer le cahier de charge, mettre en commun le travail des différents groupes ou simplement pour aiser la communication, il nous a fallu utiliser plusieurs ressources qu'il est bon de présenter puisque celles-ci ont eu leur poids pour mettre le projet à bien.

Pour tester des fonctions du PCB nous avons utilisé l'espace de développement de MPLab avec le compilateur XC8. Ce choix découlait simplement de la compatibilité entre la version 6.20 de MPLab avec le microcontrôleur employé. Cependant pour écrire le code en C, les membres de ce projet préféraient utiliser les logiciels de Visual Studio et Visual Code.

Un repos sur Github a été distribué à l'ensemble des groupes contenant les squelettes des fonctions à réaliser afin que tout le monde ait une idée globale des INs et OUTs de leurs modules dans l'ensemble du projet. Cela permettait aussi plus tard de pull et push plusieurs versions et garder la dernière version disponible pour tous afin de tester les compatibilités entre les modules.

Le reste de la communication se faisait selon les groupes : globalement par mails via les adresses Outlook de l'école, parfois par discord ou Messenger pour des individus en particulier.

3.3 Coordination (Equipe1)

3.3.1 Déroulement de la conception du projet

Pour faciliter la conception des fonctions de chaque équipe, un premier squelette du projet entier a été distribué par Github au début de la première séance. Cependant beaucoup de temps de celle-ci avait été employé par les groupes à se familiariser avec les configurations et registres de leur composant propre à travers les *data sheets* de ces derniers.

Lors de la deuxième séance beaucoup débutaient leur code, et à la moitié de celle-ci nous avons distribué ce rapport ci-présent à compléter par équipe et avec leurs parties des suggestions et des balises de points à aborder. Cela permettait aux groupes qui le désiraient de progresser en parallèle sur leur code et leur rapport.

Les deux dernières séances virent beaucoup de va-et-vient entre des codes paraissant terminés et des erreurs de compilation qui prirent un temps conséquent à résoudre.

Heureusement lors de la dernière séance au prix de quelques concessions d'optimisation au niveau de la précision de l'interruption chaque seconde (lesquelles ne seraient importantes que sur un laps de temps d'une année) le projet aboutit avec le cahier des charges respecté.

Globalement de séance en séance l'équipe de coordination se rendait disponible pour chaque question de logique ou de praticité des groupes demandant de l'aide ou se trouvant en des impasses.

3.3.2 Architecture Logicielle

L'annexe "Rapport Architecture Logicielle" de Victor décrit le fonctionnement du code et sa structure couche par couche. Voir point 7. Annexe.

3.4 RTC (Equipe2)

3.4.1 Présentation du composant

La **Real-Time Clock** (RTC) est un composant intégré qui permet de maintenir l'heure et la date avec précision, même lorsque le microcontrôleur est en mode veille ou que le système est alimenté par une source faible. Son rôle principal est de fournir un horodatage fiable pour les applications nécessitant une mesure du temps réel pour toute application embarquée qui en a besoin.

La RTC interne repose sur un oscillateur à quartz externe ou interne, dont la fréquence stable garantit la précision de la mesure du temps. Cette conception assure que les secondes, minutes, heures, jours, mois et années sont conservés de manière fiable dans des registres dédiés, accessibles directement par le microcontrôleur.

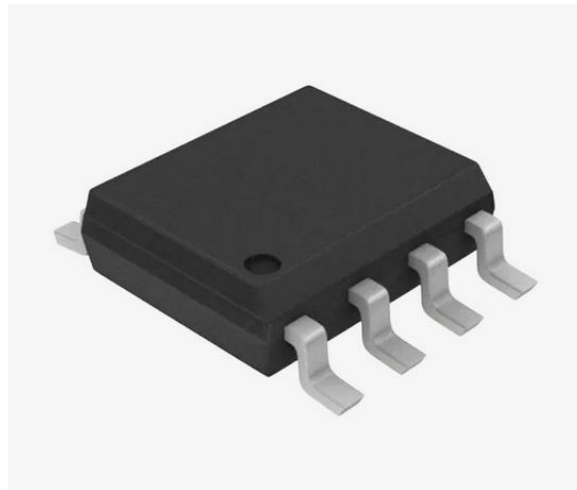


Image 1 – RTC DS1307

3.4.2 Présentation du module RTC

Intégration au PCB

La RTC est intégré sous forme d'un circuit intégré soudé sur le PCB. Le module est alimenté par la tension principale du microcontrôleur. Les registres internes stockent toutes les informations et permettent de gérer les fonctionnalités avancées comme les alarmes et les timers périodiques.

Registres et flags.

La RTC du PIC18F26K83 contient plusieurs **registres** dédiés : les secondes, minutes, heures, jour de la semaine, date, mois, année. Un registre de contrôle permet de gérer les alarmes, les timers périodiques, et de vérifier l'état du module. Les flags associés à ce registre permettent de savoir si une alarme a été déclenchée, si le compteur a débordé, ou si l'oscillateur fonctionne correctement. Ces registres sont accessibles directement par le microcontrôleur, ce qui permet la lecture et l'écriture rapides des informations temporelles.

Communication et fonctionnement.

La communication avec la RTC s'effectue via l'interface **I2C**, ce qui permet au microcontrôleur de lire et d'écrire les registres du module sans composants supplémentaires. Cette interface série bidirectionnelle garantit un échange de données fiable entre le microcontrôleur et la RTC. Le module peut également générer des interruptions sur le microcontrôleur pour notifier des événements précis, comme le déclenchement d'une mesure ou l'exécution d'une routine planifiée. Les timers intégrés à la RTC permettent de programmer des déclenchements réguliers, tandis que les alarmes programmables peuvent activer des routines à des heures ou jours spécifiques, offrant ainsi une grande flexibilité pour les systèmes autonomes.

Utilisation dans notre projet.

Dans le cadre de notre projet, la RTC a été utilisée pour récupérer l'heure et la date et les afficher sur l'écran LCD. L'utilisateur peut également modifier chaque élément de l'heure ou de la date individuellement, en modifiant les registres correspondants, ce qui offre un contrôle précis sur les secondes, minutes, heures, jours, mois et années.

3.4.3 Code RTC

Le fichier **rtc_ds1307.c** assure la gestion du module RTC. Il permet d'initialiser le composant, de communiquer avec lui via le bus I2C, ainsi que de lire et d'écrire l'heure et la date. Les fonctions implémentées fournissent une interface simple.

Initialisation de la communication I2C

La communication avec la RTC s'effectue via le bus I2C. Le fichier s'appuie sur le pilote `i2c_bus`, qui encapsule les transactions I2C.

Les adresses I2C du DS1307 sont définies sous forme de constantes pour les opérations d'écriture et de lecture, conformément à la documentation du composant.

Méthodes de conversion

Le DS1307 stocke les données temporelles au format BCD (Binary Coded Decimal). Afin de faciliter l'utilisation des valeurs dans le programme, deux fonctions utilitaires ont été implémentées.

- La fonction **bcd_to_dec()** permet de convertir une valeur BCD en valeur décimale exploitable par le programme.
- La fonction **dec_to_bcd()** réalise l'opération inverse, nécessaire avant l'écriture des données dans les registres du RTC.

Ces fonctions sont utilisées systématiquement lors des opérations de lecture et d'écriture afin de garantir la cohérence des données.

Fonctions d'accès aux registres.

Deux fonctions internes permettent de simplifier l'accès aux registres de la RTC :

- **Rtc_read_register()** lit la valeur d'un registre donné du DS1307 via une transaction I2C.
- **Rtc_write_register()** écrit une valeur dans un registre spécifique de la RTC.

Ces fonctions isolent la logique I2C et améliorent la lisibilité du code principal.

Fonction **rtc_init()**

La fonction **rtc_init()** permet d'initialiser le module RTC au démarrage du système. Elle commence par vérifier la présence du composant en tentant de lire le registre des secondes. Si cette opération échoue, une erreur est retournée, indiquant un problème de communication ou l'absence du composant.

Ensuite la fonction vérifie l'état du bit CH (Clock Halt). Si ce bit est activé, l'oscillateur de la RTC est arrêté. Il est alors remis à zéro afin de démarrer correctement l'horloge.

Fonction **rtc_set_time()**

La fonction **rtc_set_time()** permet de régler l'heure et la date du RTC à partir d'une structure `rtc_time_t`.

Elle commence par vérifier la validité du pointeur passé en paramètre ainsi que la cohérence des valeurs (heures, minutes, jour et mois). Les valeurs sont ensuite converties du format décimal vers le format BCD avant d'être écrites dans les registres correspondants du DS1307. Lors de l'écriture des secondes, le bit CH est forcé à zéro afin de garantir que l'oscillateur reste actif.

Cette fonction assure ainsi une mise à jour fiable et sécurisée de l'horloge.

Fonction `rtc_get_time()`

La fonction `rtc_get_time()` permet de lire l'heure et la date courantes depuis la RTC. Elle réalise une lecture séquentielle des registres du DS1307 en une seule transaction I2C, ce qui améliore l'efficacité de la communication. Les valeurs lues, stockées au format BCD, sont ensuite converties en décimal avant d'être copiées dans la structure `rtc_time_t`.

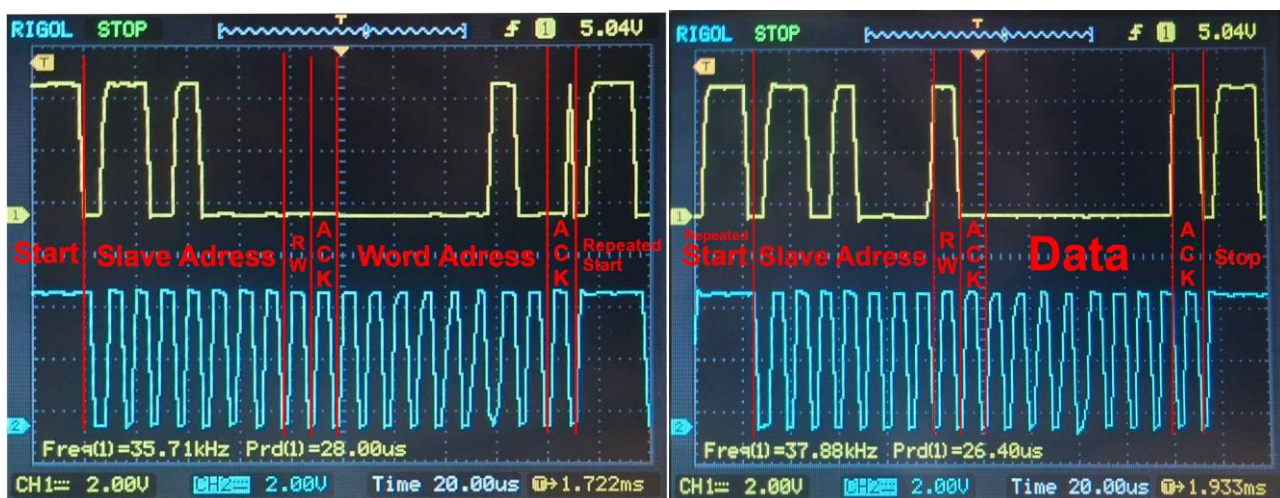
Les champs non utilisés par l'application, tels que le jour de la semaine ou l'année, sont volontairement ignorés.

3.4.4 Visualisation à l'oscilloscope

Les figures ci-dessous présentent des captures réalisées à l'oscilloscope du bus I²C reliant le microcontrôleur à la RTC **DS1307**.

La voie **CH1 (jaune)** correspond à la ligne **SDA (données)** et la voie **CH2 (bleu)** correspond à la ligne **SCL (horloge)**.

Le protocole I²C repose sur une communication synchrone maître-esclave : le microcontrôleur agit comme **maître** et génère l'horloge SCL, tandis que la RTC DS1307 agit comme **esclave** et répond aux requêtes.



Décomposition de la trame I²C observée

Condition START

La communication débute par une **condition START**, identifiable par un passage de SDA de l'état haut à bas **alors que SCL est à l'état haut**.

Cette transition indique à tous les périphériques présents sur le bus qu'une nouvelle communication va commencer.

Envoi de l'adresse esclave + bit R/W

Après la condition START, le microcontrôleur envoie l'**adresse I²C de la RTC DS1307 sur 7 bits (110 1000), suivie du bit R/W** :

- 0 → écriture (Write)
- 1 → lecture (Read)

Dans la première trame observée, le bit R/W est à 0, ce qui signifie que le maître souhaite **écrire** dans la RTC.

Cette étape permet de sélectionner le composant cible sur le bus.

Bit d'acquittement (ACK)

Après l'envoi de l'adresse, la RTC force la ligne SDA à 0 pendant un cycle d'horloge pour indiquer un **ACK**.

Cet acquittement confirme que l'esclave a bien reconnu son adresse et qu'il est prêt à communiquer.

Envoi de l'adresse du registre (Word Address)

Le microcontrôleur transmet ensuite l'**adresse interne du registre** du DS1307 à accéder (par exemple : secondes, minutes, heures, etc.).

Cette adresse est également suivie d'un **ACK** de la part de la RTC.

Cette étape positionne le pointeur interne de la RTC sur le registre souhaité.

Dans cette trame, on souhaite accéder au registre des heures :

Word Address : 000 0010 (binaire) = 02 (hexadecimal) → **Registre de heures de la RTC**

Condition Repeated START

Une **condition Repeated START** est ensuite générée sans libérer le bus (pas de STOP intermédiaire).

Elle est reconnaissable par une nouvelle transition SDA haut → bas avec SCL à l'état haut.

Cette séquence est typique d'une **lecture après écriture**, très courante avec les mémoires et les RTC :

- écriture de l'adresse du registre
- puis lecture de son contenu

Envoi de l'adresse esclave + bit Read

Après le Repeated START, l'adresse de la RTC est renvoyée, cette fois avec le bit **R/W = 1**, indiquant une **lecture**.

La RTC acquitte de nouveau par un ACK.

Transmission des données

La RTC transmet ensuite les **données du registre sélectionné** (heure, minutes, secondes, etc.), codées au format **BCD**, conformément à la documentation du DS1307.

Chaque octet transmis est synchronisé sur les fronts d'horloge SCL.

Après la réception de l'octet, le microcontrôleur génère un ACK pour indiquer qu'il souhaite poursuivre la lecture.

Dans cette trame, on peut constater que la RTC nous retourne un 0 pour l'heure actuelle.

Data : 0000 0000 (BCD) = 0 (decimal) → La RTC est réglée à minuit

Condition STOP

La communication se termine par une **condition STOP**, caractérisée par un passage de SDA de l'état bas à haut **alors que SCL est à l'état haut**.

Cette condition libère le bus I²C et marque la fin de la transaction.

3.5 Menu LCD (Equipe2)

3.5.1 Présentation du composant

Un **écran LCD** (Liquid Crystal Display) est un composant électronique permettant d'afficher des informations textuelles ou graphiques en modulant la lumière transmise à travers des cristaux liquides. Un écran LCD peut afficher plusieurs caractères ou icônes simultanément, ce qui l'a rendu idéal pour représenter un menu interactif dans ce projet. L'écran utilisé est un modèle 16x2, avec un rétroéclairage intégré pour une meilleure lisibilité.

Son rôle principal dans notre projet est de fournir une interface visuelle à l'utilisateur, permettant donc de naviguer dans le menu, sélectionner des options, visualiser des états ou consulter des informations telles que l'heure fournie par la RTC, ou des mesures provenant des capteurs.

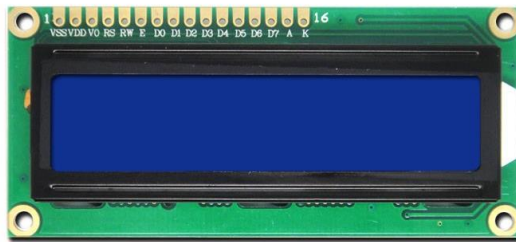


Image 2 - Ecran LCD

3.5.2 Code Menu

Le menu de l'application a été implémenté afin de permettre à l'utilisateur d'interagir simplement avec le système à l'aide de boutons poussoirs et de l'écran LCD. Le code du menu est organisé de manière modulaire, ce qui facilite la lisibilité et la maintenance.

Le fichier **menu.c** centralise toute la logique liée à la navigation, à l'affichage et à l'édition des paramètres. Il s'appuie sur plusieurs modules externes, notamment le pilote de l'écran LCD, la gestion des boutons, la RTC, les capteurs BMP280 et SHT30 ainsi que le module de datalogger.

Le menu est basé sur une machine à états, ce qui permet de gérer les différentes phases d'utilisation de l'interface.

Les états permettent de distinguer :

- L'affichage du menu principal,
- La navigation dans les sous-menus,
- L'affichage des données,
- L'édition des paramètres.

Cette approche rend le comportement du menu plus clair et évite les erreurs liées à des conditions complexes.

Communication avec les autres modules.

Le rôle principal du menu, en ce qui concerne la communication avec les autres modules tels que le SHT30 et BMP280, a été d'assurer l'interface entre l'utilisateur et ces différents composants. Le menu s'appuie sur les fonctions fournies par chaque groupe pour récupérer les valeurs mesurées. Ces données sont ensuite mises en forme et affichées sur l'écran LCD dans des sous-menus dédiés, permettant à l'utilisateur de consulter facilement les informations.

En ce qui concerne le **datalogger**, le menu permet de le configurer et d'accéder à ses paramètres. L'utilisateur peut modifier ces paramètres directement depuis l'interface, et le menu transmet ensuite les nouvelles valeurs au datalogger via ses fonctions publiques. Cette organisation garantit que le menu reste uniquement une interface, sans intervenir dans le traitement interne des données ou la logique d'enregistrement.

Ainsi, les sous-menus disponibles sont :

- **Date et Heure** : Permet l’affichage et la modification de l’heure et de la date.
- **Pression & Temp** : Affiche la pression atmosphérique et la température mesurées par le capteur BMP280.
- **Humidité & Temp** : Affiche l’humidité relative et la température mesurées par le capteur SHT30.
- **Datalogger Cfg** : Permet de configurer le datalogger : période d’échantillonnage, état (démarré/arrêté).
- **Clear EEPROM** : Efface toutes les données stockées dans la mémoire EEPROM.

Structure de contexte du menu

La structure **menu_context_t** permet de conserver l’état actuel du menu. Elle stocke notamment l’état courant du menu, l’index de l’élément sélectionné, le sous-menu actif, le champ en cours d’édition. Grâce à cette structure, toutes les informations nécessaires au fonctionnement du menu sont centralisées, ce qui simplifie la gestion des transitions entre les différents écrans.

```
typedef struct {  
    menu_state_t state;  
    uint8_t main_index;           // Index dans le menu principal  
    uint8_t submenu_index;       // Index dans le sous-menu actuel  
    submenu_type_t current_submenu;  
    uint8_t scroll_offset;        // Offset pour le scroll horizontal  
    uint8_t editing;             // Mode édition activé  
    datetime_field_t edit_field; // Champ en cours d’édition  
    datalogger_field_t dl_edit_field; // Champ datalogger en cours  
    uint8_t edit_cursor;         // Position du curseur en édition  
} menu_context_t;
```

Image 3 – Contexte du Menu

Fonctions principales.

Fonction menu_init()

La fonction **menu_init()** permet d’initialiser le menu au démarrage du système. Elle réinitialise l’état du menu, remet les index à zéro et prépare l’écran LCD pour le premier affichage. Cette fonction est appelée une seule fois lors de l’initialisation du microcontrôleur.

Fonction menu_update()

La fonction **menu_update()** constitue le coeur de la logique du menu.

Elle est appelée périodiquement dans la boucle principale du programme et permet :

- De détecter les appuis sur les boutons,
- De mettre à jour l’état du menu,

- De gérer les transitions entre les différents écrans.

Elle renvoie une valeur indiquant si l’affichage doit être rafraîchi, ce qui permet d’optimiser l’utilisation de l’écran LCD.

Fonction menu_display()

La fonction **menu_display()** est chargée de l’affichage sur l’écran LCD.

En fonction de l’état courant du menu, elle appelle les fonctions d’affichages appropriées :

- Menu principal,
- Sous-menu,
- Affichage des capteurs,
- Mode édition.

Cette séparation entre logique et affichage rend le code plus clair et plus facile à modifier.

Logique de scroll pour les titres.

Afin d’avoir une vision plus claire des titres du menu, une logique de **scroll** a été implémentée dans le code du menu. Celle-ci consiste à faire défiler les titres afin qu’ils apparaissent en entier, malgré l’écran LCD limité à 16 caractères. Cette fonctionnalité est principalement implémentée dans les fonctions d’affichage, notamment lors de l’écriture des lignes du menu principal.

Lorsqu’un élément est sélectionné, le code compare la longueur du texte à l’espace réellement disponible sur l’écran. Si le texte dépasse cette limite, un décalage progressif est appliqué grâce à une variable de décalage (scroll offset). Ce décalage permet d’afficher successivement différentes portions du texte, donnant l’impression d’un défilement horizontal.

La mise à jour du défilement est pilotée par un compteur temporel incrémenté périodiquement. A chaque dépassement d’un délai défini, le décalage est incrémenté, puis réinitialisé une fois la fin du texte atteinte. Cette approche permet d’obtenir un défilement fluide et lisible, tout en évitant de bloquer l’exécution du programme. La logique de scroll est ainsi intégrée de manière non bloquante et s’adapte automatiquement à la longueur des titres du menu.

Affichage et modification de l’heure et de la date.

Le menu permet à l’utilisateur d’afficher et de modifier l’heure et la date directement depuis l’interface, à l’aide des boutons poussoirs et de l’écran LCD. Lorsqu’un utilisateur sélectionne l’option correspondante, le menu récupère d’abord la valeur actuelle de la RTC et la copie dans une structure temporaire dédiée à l’édition. Cette approche permet d’effectuer les

modifications sans impacter immédiatement l'horloge interne.

La modification s'effectue champ par champ, par incrémentation ou décrémentation des valeurs (jour, mois, heure, minute). Les limites minimales et maximales de chaque champ sont prises en compte afin d'éviter toute valeur incohérente.

```
// Obtient le maximum pour un champ de date/heure
static uint8_t get_field_max(datetime_field_t field) {
    switch (field) {
        case FIELD_DAY:    return 31;
        case FIELD_MONTH:  return 12;
        case FIELD_YEAR:   return 99;
        case FIELD_HOUR:   return 23;
        case FIELD_MINUTE: return 59;
        case FIELD_SECOND: return 59;
        default:            return 0;
    }
}

// Obtient le minimum pour un champ de date/heure
static uint8_t get_field_min(datetime_field_t field) {
    switch (field) {
        case FIELD_DAY:
        case FIELD_MONTH: return 1;
        default:           return 0;
    }
}
```

Image 4 – Fonctions des champs numériques

Pendant la phase d'édition, le champ en cours de modification est mis en évidence par un clignotement à l'écran, ce qui améliore la lisibilité et l'ergonomie de l'interface.

Une fois les modifications validées par l'utilisateur, les nouvelles valeurs sont écrites dans les registres de la RTC. En cas d'annulation, les modifications ne sont pas prises en compte et l'horloge conserve ses valeurs initiales. Cette méthode garantit une modification sécurisée et contrôlée de l'heure et de la date.

3.6 EEPROM et DATALOGGER (Equipe5)

3.6.1 Présentation du composant EEPROM

Le M93C66 est une mémoire non volatile de type EEPROM série, généralement intégrée sur un circuit imprimé (PCB) pour stocker des données persistantes.

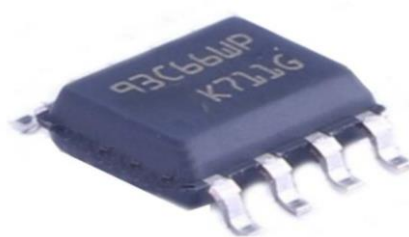


Image 5 - EEPROM

Rôle de la mémoire

Le M93C66 permet de conserver des informations même en l'absence d'alimentation. Sa capacité typique est de 4096 bits, soit 512 octets.

Cette mémoire est utilisée pour stocker des données de configurations aussi bien que des données fournies par des capteurs.

Rôle du périphérique de communication

La communication avec le M93C66 s'effectue via une interface série Microwire, reposant sur plusieurs lignes :

- CS (Chip Select) : activation du composant,
- SK (Serial Clock) : synchronisation des échanges,
- DI (Data Input) : données envoyées vers la mémoire,
- DO (Data Output) : données lues depuis la mémoire.

Sur le PCB, le M93C66 agit comme un périphérique esclave, piloté par un microcontrôleur maître. Celui-ci gère l'horloge, la sélection du composant et les trames de commande (lecture, écriture, effacement).

Ce mode de communication série permet de réduire le nombre de pistes, d'optimiser la surface du PCB et de simplifier l'intégration matérielle.

3.6.2 Le Protocole de communication

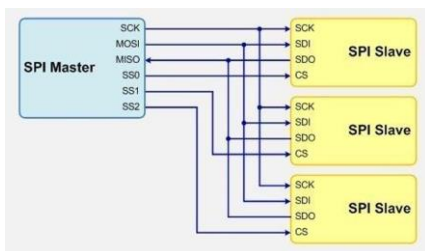


Image 6 - Schéma du protocole SPI

La communication avec le module Eeprom se fait via le protocole SPI (Serial Peripheral Interface), consistant en une relation maîtresse et plusieurs esclaves. Le microcontrôleur joue le rôle de maître et pilote plusieurs périphérique dit esclaves.

Chacun de ces esclaves est différencié par une liaison SS (Slave Select) propre à son maître. Cette ligne permet au maître de choisir le composant avec lequel il souhaite communiquer laissant ainsi inactif les autres esclaves sur le bus.

La communication est synchrone et full-duplex : les données peuvent être transmises et reçues simultanément.

Trois connexions communes avec tous les esclaves sont nécessaires :

- SDI (MOSI = Master Out slave In) : données envoyées du maître vers l'esclave
- SDO (MISO = Master In slave Out) : données envoyées de l'esclave vers le maître
- SCK = Serial Clock : signal d'horloge généré par le maître.

3.6.3 Compression des mesures

L'EEPROM ayant une capacité limitée de 512 octets, il est important d'optimiser l'écriture des données dans ce dernier, et les compresser dans une taille limitée.

Nous avons donc pris la décision de compresser les mesures de température, d'humidité et de pression à un seul octet.

Cette compression entraîne naturellement une perte de résolution mais nous avons calculé ces pertes en amont afin de nous assurer que les mesures demeurent tolérables.

A) Température

La température est renseignée par le capteur SHT30 sous la forme de 2 octets. Celle-ci est exprimée sous la forme d'un entier correspondant à 100 fois la température en degrés Celsius.

Nous avons choisi de compresser les données tel que nous puissions mesurer la température entre -40°C et 87,5°C avec une résolution de 0,5°C.

Pour ce faire, nous prenons la valeur en 2 octets renseignée par le capteur et y soustrayons l'offset établi à -40°C (ce qui revient donc à additionner par 4000)

-> $\text{shifted_t16} = t16 + 4000$

Avant d'obtenir une résolution de 0,5°C, nous appliquons la formule $(n + (d / 2u)) / d$ où n représente la température et d représente 100 fois la résolution (ici d=50)

Nous obtenons ainsi une température stockée sur un octet.

B) Humidité

L'humidité est renseignée par le capteur SHT30 sous la forme de 2 octets. Celle-ci est exprimée sous la forme d'un entier correspondant à 100 fois le pourcentage d'humidité.

La plage de mesure que nous avons choisie est la même que celle prise en compte par le capteur, à savoir de 0 à 100%. Nous allons seulement modifier la résolution de la mesure pour obtenir une résolution de 0,5%.

Il suffit d'appliquer la formule $(n + (d / 2u)) / d$ avec d=50 pour stocker l'humidité sur un octet.

C) Pression

La pression est renseignée par le capteur BMP280 sous la forme de 4 octets. Celle-ci est exprimée sous la forme d'un entier correspondant à la pression en Pascal avec une résolution de 50Pa.

Nous allons d'abord convertir cette valeur en hPa afin de pouvoir la stocker correctement sur un seul octet en appliquant la formule $(n + (d / 2u)) / d$ avec d=100

Nous définissons ensuite une plage de mesure entre 800hPa et 1200hPa avec un offset établi à 800hPa.

Nous soustrayons donc 800 à la valeur précédemment obtenue, avant de lui appliquer la formule $(n + (d / 2u)) / d$ avec d=2.

Nous obtenons donc une valeur en hPa correspondant à la pression diminuée de 800 avec une résolution de 2hPa.

3.6.4 Formatage de la mémoire

Pour maximiser le stockage, chaque donnée (température, humidité et pression) sont stockées sur un octet.

La mémoire sera composée des valeurs utiles en début puis des paquets de 3 octets (température, pression, humidité) qui se suivent jusqu'à la fin de la mémoire.

Les valeurs en début de mémoire sont :

- Le *Sample Period* qui donne l'indication de combien de minutes il se passe entre les mesures.
- Le jour de la première mesure
- Le mois de la première mesure
- L'heure de la première mesure
- La minute de la première mesure
- Le nombre de mesures qui ont été faites
- Une variable booléenne qui indique si la routine de mesure est en cours (1) ou non (0)

Toutes ses variables sont contenues sur 1 octet, la variable booléenne aurait pu être enregistrée sur un seul bit mais cela n'aurait pas augmenté le stockage et aurait rendu le datalogger plus complexe. Les heures, les minutes sont enregistrées en DCB.

Data Logger	
Configuration	Delta T
	Jour de la 1ère mesure
	Mois de la 1ère mesure
	Heure de la 1ère mesure
	Minute de la 1ère mesure
	Nombre de mesure
	IsRunning
LOG	Température
	Pression
	Humidité
	Température
	Pression
	Humidité
	Température
	Pression
	Humidité

Tableau 2 - Représentation du Datalogger

3.6.5 Code

Le menu permet d'initialiser une valeur de temps entre chaque mesure grâce à la fonction *set_sample_period*. Ensuite, lorsque le menu commence un enregistrement, il va appeler la fonction *dl_set_running* qui va initialiser les valeurs de dates, remettre à 0 le compteur et mettre à 1 la variable booléenne qui indique que la routine de mesure est en cours. Toutes ces variables sont aussi sauvegardées en variable globale dans la RAM du microcontrôleur ce qui permet de ne pas devoir faire des requêtes de lecture permanente à l'EEPROM. Pour ce faire, il y a la méthode *cache_update_field* qui permet de mettre à jour la variable en cache et de l'écrire dans l'EEPROM. Ensuite à chaque intervalle de temps, les 3 données reçues par les capteurs sont réduites sur un octet et enregistré dans l'EEPROM. On utilise pour cela les fonctions *sensor_reduce* et *dl_push_record*.

3.6.6 Visualisation à l'oscilloscope

WRITE	Write Data to Memory	1	01	A8-A0	D7-D0	20
-------	----------------------	---	----	-------	-------	----

La trame de la communication SPI est constitué d'un cycle de 20bits.
Le 1e correspond au **bit de start** toujours égal à "1". Suivi d'un **OP-code** "01"(write). L'**adresse** correspond au 9 bits suivant (adresse 1 de l'Eeprom) et terminé par la **donnée** sur 8bits (valeur 200)
1.01.00000001.11001000

- En jaune = trame SPI
- En bleu = trame de CLK (clock)

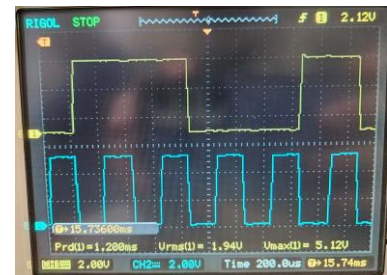
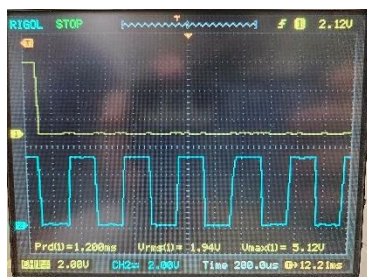
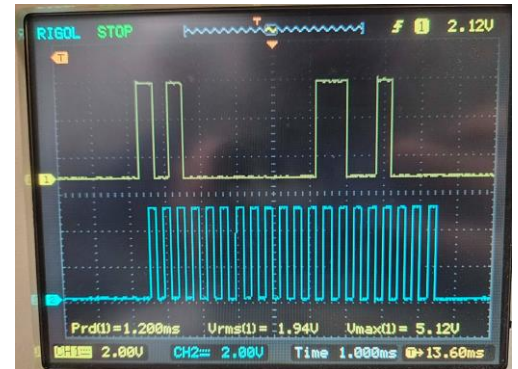


Image 7 - Communication SPI : Trame à l'oscilloscope

3.7 Température ambiante et Humidité Relative (Equipe4)

3.7.1 Présentation du capteur SHT30

Le SHT30 est un capteur numérique permettant de mesurer la température et l'humidité. Il intègre un capteur de température en silicium destiné à la mesure de la température ambiante, un capteur capacitif pour la mesure de l'humidité relative, ainsi qu'un convertisseur analogique-numérique de 14 bits fournissant des valeurs numériques précises.

Le SHT30 communique avec le microcontrôleur via le bus I²C, ce qui permet de récupérer les mesures de température et d'humidité sous forme de trames numériques. L'adresse du capteur peut être 0x44 ou 0x45 selon le raccordement de sa broche ADDR.



Image 8 - Capteur SHT30

Caractéristiques principales

Température (°C)

- Plage de mesure : -40 °C à 125 °C (erreur de +- 1 °C)
- Plage de fonctionnement privilégiée : 5 °C à 60 °C (erreur de +- 0.2 °C)
- Résolution : 0.01 °C

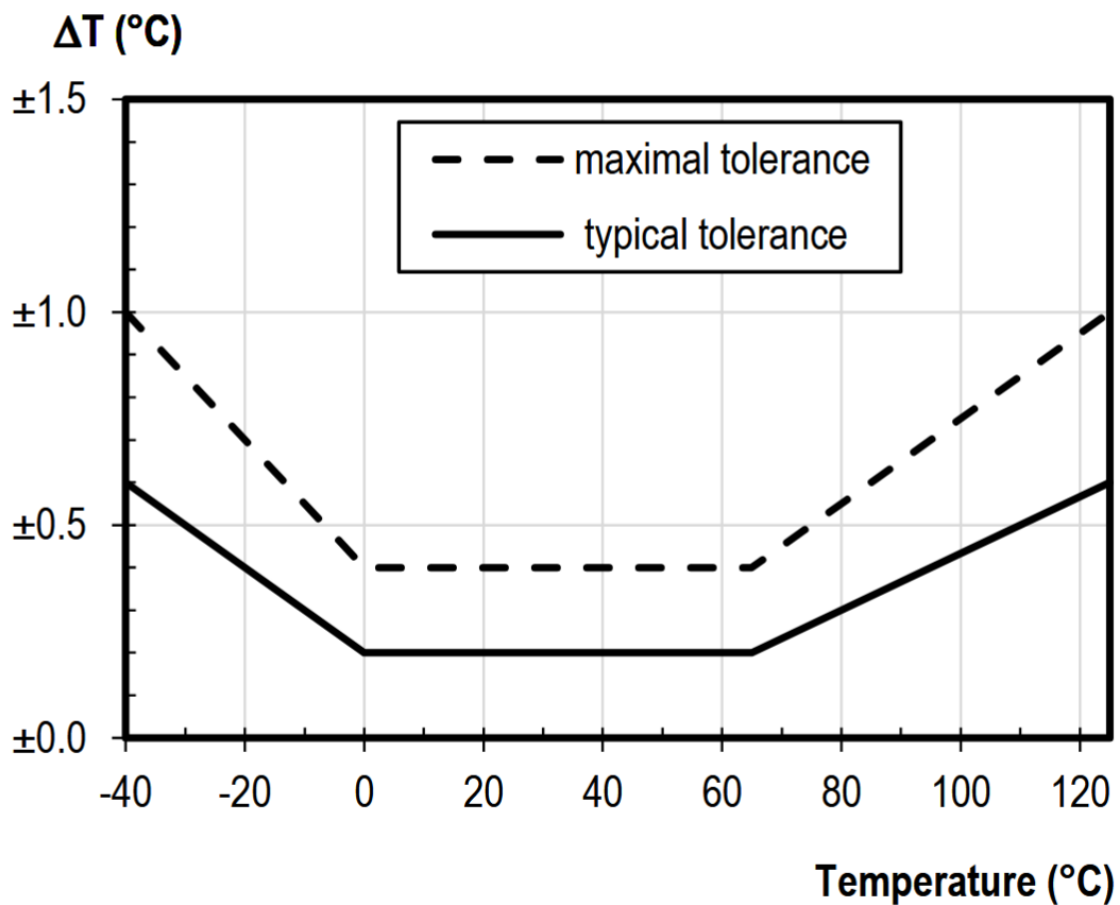


Figure 1 : Précision de la mesure de température du capteur SHT30.

(Lien légende

https://sensirion.com/media/documents/213E6A3B/63A5A569/Datasheet_SHT3x_DIS.pdf)

Comme illustré à la figure 1, le capteur SHT30 présente une réponse linéaire principalement dans la plage comprise entre 0 °C et 65 °C. En dessous de 0 °C, l'erreur de mesure augmente progressivement de manière quasi linéaire, s'écartant légèrement de la précision nominale annoncée ($\pm 0,3$ °C). De même, lorsque la température dépasse 65 °C, l'erreur croît également. Conformément aux recommandations du fabricant, il est donc conseillé d'utiliser le capteur dans une plage comprise entre 5 °C et 60 °C.

Selon le fabricant, « le capteur montre les meilleures performances lorsqu'il est exploité dans une gamme de température et d'humidité normales ». Aucune valeur chiffrée précise n'étant fournie, cette indication laisse entendre que le capteur n'est pas destiné à des environnements présentant des conditions extrêmes. On peut donc supposer que cette plage dite « normale » correspond à celle pour laquelle l'erreur de mesure reste maîtrisée.

Dans le cadre de notre projet, les bilans climatologiques hivernaux publiés par l'Institut royal météorologique de Belgique (<https://www.meteo.be/fr/climat/climat-de-la-belgique/bilans-climatologiques/2024/hiver> et <https://www.meteo.be/fr/climat/climat-de-la-belgique/bilans-climatologiques/2025/hiver>) montrent que les températures minimales descendent parfois en dessous de 0 °C, avec -6,8 °C le 10 janvier 2024 et -4 °C le 18 février 2025. Dans ces conditions, l'erreur du SHT30 augmente légèrement, mais elle reste faible et acceptable pour une petite station météorologique.

Ainsi, même si le SHT30 pourrait être légèrement moins précis en hiver, il reste parfaitement utilisable pour notre projet. Pour garantir la cohérence et la simplicité du traitement des données avec l'équipe 5 (EEPROM et DATALOGGER), nous avons décidé de stocker et gérer les valeurs de température provenant du SHT30, en assurant leur exploitation uniforme au sein du DATALOGGER.

Humidité relative (RH%)

- Plage de mesure : 0% à 100% (erreur de +- 4%)
- Plage de fonctionnement privilégiée : 20% à 80% (erreur de +-2%)
- Résolution : 0,006 % RH

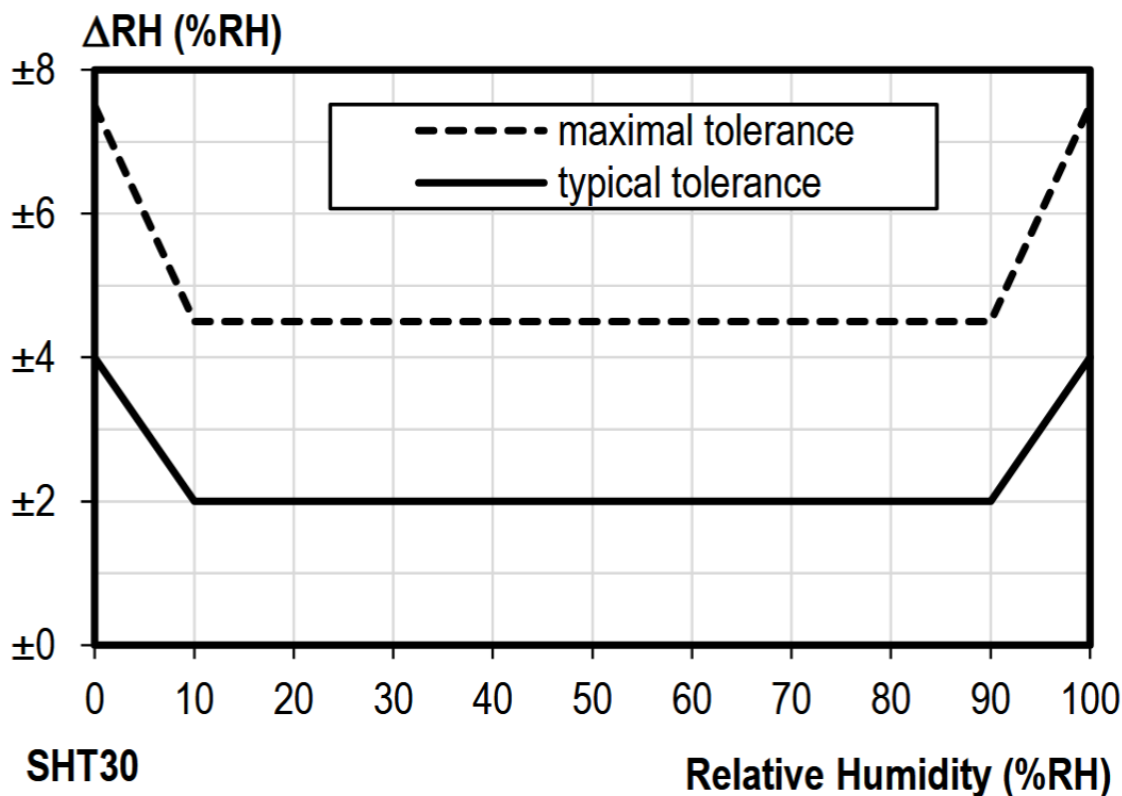


Image 9 - Tolérance de la mesure d'humidité relative à 25 °C pour le capteur SHT30.

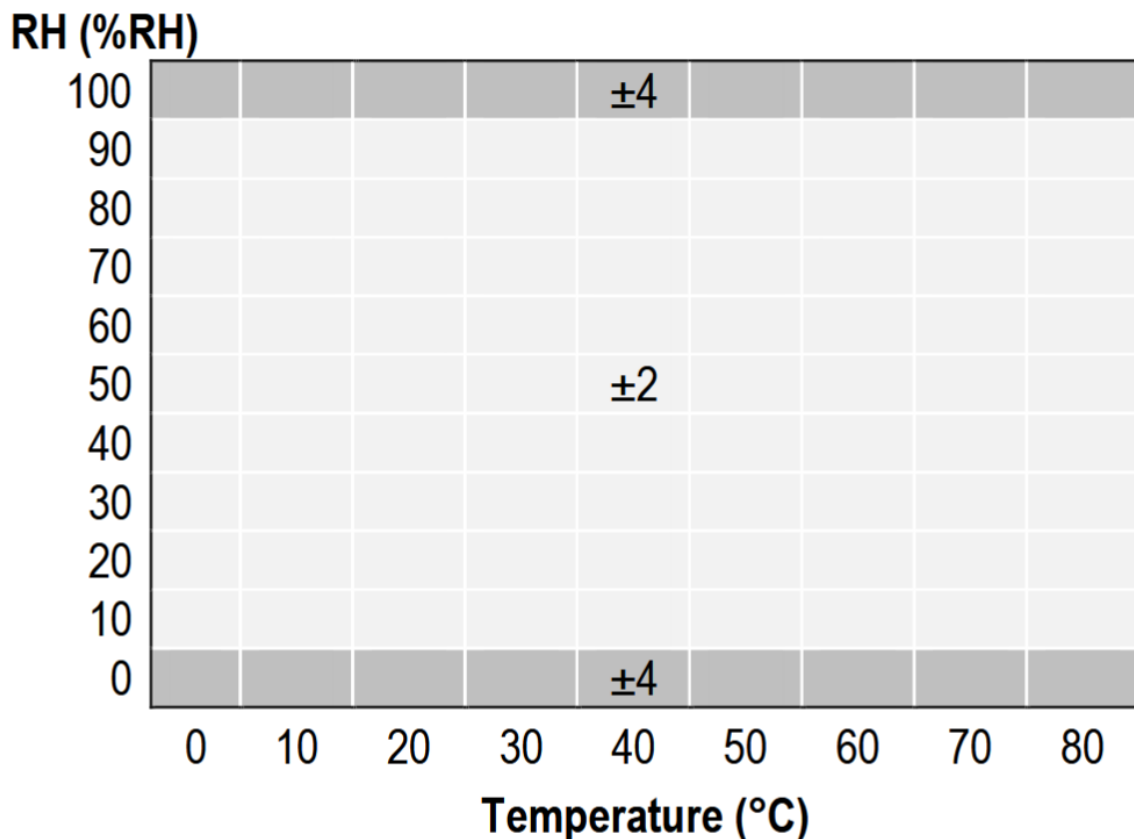


Image 10 - Tolérance typique de la mesure d'humidité relative selon la température pour le capteur SHT30.

Comme illustré aux figures 2 et 3, le capteur SHT30 présente une précision optimale pour la mesure de l'humidité relative dans la plage comprise entre 10 %RH et 90 %RH, où l'erreur reste d'environ ± 2 %RH. En dehors de cette plage, c'est-à-dire en dessous de 10 %RH ou au-dessus de 90 %RH, l'erreur de mesure augmente progressivement pour atteindre ± 4 %RH.

Selon le fabricant, le capteur offre les meilleures performances lorsqu'il est exploité dans une gamme d'humidité dite « normale ». Aucune valeur chiffrée précise n'étant fournie, on peut supposer que cette plage correspond à celle pour laquelle l'erreur reste maîtrisée et acceptable pour une application standard.

Dans le cadre de notre projet de station météorologique, le SHT30 est le seul capteur utilisé pour mesurer l'humidité relative. Bien que l'humidité relative ne soit pas strictement nécessaire pour obtenir une mesure fiable de la température, elle constitue une information importante pour l'étude des conditions climatiques et est exploitable par l'EEPROM et le DATALOGGER.

3.7.2 Protocole I²C

Dans ce chapitre, nous conservons la terminologie « maître-esclave(s) » telle qu'utilisée en cours, et non la terminologie actuelle « contrôleur-périphérique » (controller-target), par habitude et pour faciliter la compréhension par l'ensemble des membres de l'équipe. Cette convention sera également utilisée dans les paragraphes suivants.

Le protocole I²C repose sur deux lignes de communication : la ligne SCL, dédiée au signal d'horloge, et la ligne SDA, utilisée pour l'échange des données. Ces deux lignes sont partagées entre l'ensemble des périphériques connectés au bus, ce qui rend ce protocole particulièrement économique en termes de câblage. L'I²C fonctionne selon une organisation maître-esclave : le maître, généralement un microcontrôleur, contrôle entièrement la communication en générant l'horloge, en initiant les échanges et en sélectionnant l'esclave à interroger au moyen de son adresse. Les esclaves, quant à eux, ne répondent que lorsqu'ils sont sollicités par le maître.

La communication est synchrone, ce qui signifie que l'émission et la réception des données suivent strictement le rythme imposé par le signal d'horloge SCL généré par le maître. Chaque bit transmis est lu ou écrit au moment précis du front d'horloge correspondant, ce qui garantit que les données sont correctement interprétées par le récepteur, indépendamment de la vitesse interne des composants. Cette synchronisation stricte évite toute erreur de transmission, car ni le maître ni l'esclave ne traitent de données en dehors du rythme de l'horloge.

La ligne SDA fonctionne en mode semi-duplex : les données peuvent être transmises soit du maître vers l'esclave, soit de l'esclave vers le maître, mais jamais simultanément dans les deux sens. L'alternance entre lecture et écriture, coordonnée par l'horloge, constitue le cœur du protocole I²C et permet une communication fiable entre plusieurs périphériques sur le même bus.

Une transaction débute toujours par une condition de départ, caractérisée par un changement d'état de la ligne SDA alors que la ligne SCL est à l'état haut, et se termine par une condition d'arrêt définie de manière similaire. Entre ces deux conditions transitent l'adresse de l'esclave, les données échangées ainsi que les bits d'acquiescement.

Pour le SHT30, après qu'une mesure a été effectuée, le maître peut lire les résultats en envoyant une condition de départ suivie d'un entête de lecture I²C. Le capteur répond par un acquiescement (ACK) pour signaler la bonne réception de l'entête. Les octets de données sont ensuite transmis dans l'ordre suivant : deux octets de température, un octet de contrôle CRC (Code de Redondance Cyclique) pour vérifier l'intégrité des données, deux octets d'humidité relative, puis un second octet CRC.

Après chaque octet reçu, le microcontrôleur doit envoyer un ACK pour indiquer au capteur de continuer l'envoi des données. Si le capteur ne reçoit pas cet acquiescement, il n'enverra pas les octets suivants. Une fois toutes les données nécessaires lues, le maître envoie un non-acquiescement (NACK) suivi d'une condition d'arrêt (STOP) pour clore la transaction. Le maître peut également interrompre la lecture avec un NACK après n'importe quel octet si certaines données ne sont pas nécessaires, par exemple les octets CRC, ce qui permet de gagner du temps lors du traitement.

Enfin, le SHT30 peut utiliser ou non le mécanisme de blocage de l'horloge (clock stretching) selon la commande envoyée. Dans ce mode, le capteur peut maintenir la ligne SCL à l'état bas pendant qu'il effectue sa mesure, puis la relâcher une fois la conversion terminée afin de commencer l'envoi des données. Ce mécanisme garantit que le maître ne lit jamais de données incomplètes.

3.7.3 Configuration initiale du capteur

Après son alimentation, le SHT30 entre automatiquement en mode d'attente et devient prêt à recevoir des commandes via l'I²C. Pour l'utiliser dans de bonnes conditions, il est nécessaire de sélectionner, parmi les paramètres proposés dans sa documentation technique, ceux qui conviennent le mieux à l'application.

Le capteur peut fonctionner en mode single-shot ou en mode périodique. Dans le cadre de ce projet, le mode single-shot est privilégié : le microcontrôleur envoie une commande de mesure toutes les 15 minutes, ce qui permet de réaliser des acquisitions ponctuelles à intervalles réguliers sans maintenir le capteur actif en permanence.

Deux paramètres principaux doivent être définis : la répétabilité et la fréquence de mesure. La répétabilité peut être réglée sur faible, moyenne ou élevée. Un niveau élevé offre une mesure plus stable et moins bruitée, mais augmente le temps de conversion et la consommation. Étant donné que l'objectif est d'obtenir des valeurs fiables sans contrainte forte sur l'énergie, la répétabilité élevée a été retenue afin d'améliorer la précision.

En mode single-shot, la fréquence de mesure est déterminée par le microcontrôleur. Cette approche présente un double avantage : elle permet de limiter la consommation énergétique, puisque le capteur n'effectue des conversions que lorsque cela est nécessaire, et elle reste pertinente pour une station météorologique, où les conditions atmosphériques changent lentement et où une mesure toutes les 15 minutes est suffisante pour un suivi efficace.

Une fois la commande de mesure envoyée, le SHT30 effectue la conversion interne et transmet les trames contenant la température, l'humidité et leurs octets de contrôle CRC respectifs, permettant au microcontrôleur de vérifier l'intégrité des données reçues.

3.7.4 Reste du code

Le capteur SHT30 ne nécessite aucune phase d'initialisation spécifique du côté du microcontrôleur. Lorsqu'il est alimenté, il effectue automatiquement son auto-initialisation interne puis se place dans un état d'attente (*idle*). Dans cet état, il ne réalise aucune mesure mais reste à l'écoute sur le bus I²C, en attente de la réception d'une commande.

Lorsque le microcontrôleur envoie une commande de mesure (la séquence **0x24 0x00** dans notre implémentation), le capteur quitte l'état *idle*, effectue une conversion interne de la température et de l'humidité, puis se prépare à transmettre les résultats lors d'une lecture ultérieure sur le bus I²C

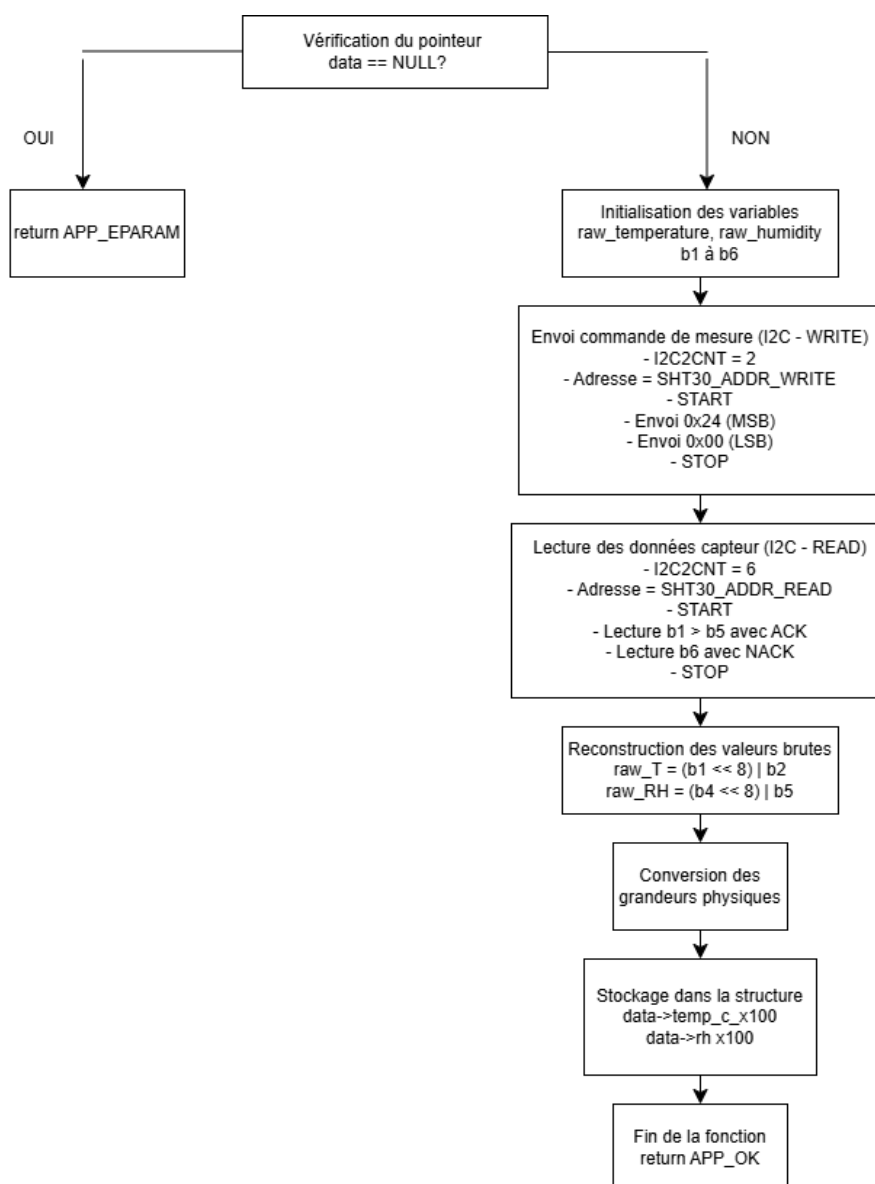


Image 11 - Organigramme de la fonction sht30_read()

Le capteur SHT30 ne nécessite aucune phase d'initialisation spécifique du côté du microcontrôleur. Lorsqu'il est alimenté, il effectue automatiquement son auto-initialisation interne, puis se place dans un état d'attente (idle). Dans cet état, il ne réalise aucune mesure mais reste à l'écoute sur le bus I²C, en attente de la réception d'une commande.

Lorsque le microcontrôleur envoie une commande de mesure (séquence 0x24 0x00 dans notre implémentation), le capteur quitte l'état idle, effectue une conversion interne de la température et de l'humidité, puis se prépare à transmettre les résultats lors d'une lecture ultérieure sur le bus I²C.

La fonction sht30_read() assure la lecture complète des données du capteur. Elle est structurée en trois étapes principales.

1. Déclenchement de la mesure :

Le microcontrôleur initie une communication I²C en mode écriture afin de déclencher une mesure interne du capteur. Le registre I2C2CNT est configuré pour transmettre deux octets correspondant à la commande de mesure. L'adresse du capteur en écriture (0x88) est ensuite chargée.

La communication débute par l'émission d'une condition de départ (START), suivie de l'envoi du premier octet de la commande (poids fort 0x24). Une fois le buffer d'émission libéré, le second octet (0x00) est transmis. Lorsque l'ensemble de la commande a été correctement envoyé, la fin de la transaction est détectée par la génération d'une condition d'arrêt (STOP), signalée par la mise à 1 du flag PCIF.

Cette séquence permet de lancer la conversion interne de la température et de l'humidité au niveau du SHT30. Après l'envoi de la commande, le programme impose un délai de 15 ms, géré par une temporisation logicielle, afin de laisser au capteur le temps nécessaire pour effectuer la conversion.

2. Lecture des données :

Une fois ce temps écoulé, le microcontrôleur initie une seconde communication I²C, cette fois en mode lecture. Le registre I2C2CNT est configuré à 6, correspondant au nombre d'octets transmis par le capteur : deux octets pour la température, un octet de contrôle, deux octets pour l'humidité et un dernier octet de contrôle. L'adresse du capteur en lecture (0x89) est ensuite chargée, puis une condition de départ (START) est générée.

Les octets sont reçus séquentiellement. Après chaque octet, le microcontrôleur envoie un acquittement (ACK) afin de poursuivre la communication. Les cinq premiers octets sont lus successivement. Avant la réception du sixième et dernier octet, le bit ACKCNT est positionné pour envoyer un non-acquittement (NACK), indiquant la fin de la réception. La transaction se termine lorsque la condition d'arrêt (STOP) est détectée via le flag PCIF.

3. Reconstitution et conversion des données

Une fois les six octets reçus, les données de température et d'humidité sont reconstituées à partir des octets de poids fort et de poids faible, afin d'obtenir des valeurs brutes codées sur 16 bits. Ces valeurs représentent le résultat interne du convertisseur du capteur et ne correspondent pas directement à des grandeurs physiques exploitables.

Les valeurs brutes sont ensuite converties en température (°C) et en humidité (%RH) à l'aide des formules fournies par le fabricant. Les résultats sont mis à l'échelle pour conserver une précision au centième, puis stockés dans la structure `sht30_data_t`, ce qui permet leur utilisation ultérieure dans le programme.

3.7.5 Conversion des données

Les mesures de température et d'humidité renvoyées par le capteur SHT30 ne sont pas directement exprimées sous forme de grandeurs physiques interprétables, telles que -40 à 125 °C pour la température ou 0 à 100 % pour l'humidité relative. Après l'envoi d'une commande de mesure, le capteur transmet 6 octets sur le bus I²C, dont seuls 4 octets contiennent des données utiles : deux octets représentent la valeur brute de la température et deux octets représentent la valeur brute de

l'humidité relative. Ces mots de 16 bits proviennent du convertisseur analogique-numérique interne du capteur et correspondent à des valeurs proportionnelles à l'échelle maximale des mesures possibles.

Pour obtenir des valeurs compréhensibles en degrés Celsius et en pourcentage d'humidité relative, les valeurs brutes sont converties à l'aide des formules fournies dans la documentation technique du SHT30 (cf. figure de conversion).

Formule de conversion de la température

$$T[^\circ C] = -45 + 175 \cdot \frac{S_T}{2^{16} - 1}$$

Formule de conversion de l'humidité

$$RH[\%] = 100 \cdot \frac{S_{RH}}{2^{16} - 1}$$

Afin de simplifier le traitement et le stockage des mesures dans le cadre du projet, et sur demande de l'équipe 5 et des coordinateurs, les valeurs converties sont stockées sous une forme centésimale. Cette méthode permet de représenter les données avec deux chiffres après la virgule sans utiliser de type flottant, ce qui facilite leur manipulation par le microcontrôleur et leur exploitation dans le DATALOGGER.

Par exemple :

- Une humidité relative de 45,2 % est stockée sous la forme 4520,
- Une température de 17,23 °C est stockée sous la forme 1723.

3.7.6 Oscilloscope

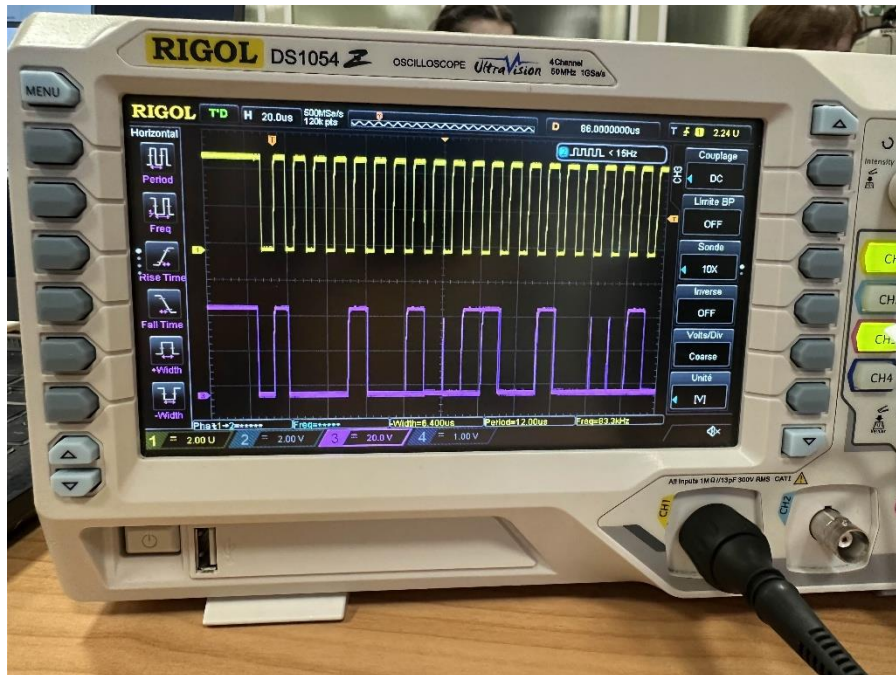


Image 12 - Communication I²C : Trames à l'oscilloscope

Afin de valider le bon fonctionnement de la communication I²C entre le microcontrôleur et le capteur SHT30, les signaux des lignes SDA (données) et SCL (horloge) ont été observés à l'aide d'un oscilloscope numérique. Ce type d'oscilloscope présente l'avantage de permettre la capture, la mise en pause et l'analyse détaillée de signaux numériques rapides, ce qui est particulièrement adapté à l'étude de protocoles de communication tels que l'I²C.

En raison de la longueur importante de la trame complète échangée entre le microcontrôleur et le capteur (commande, temporisation interne, puis lecture de plusieurs octets de données), il n'a pas été possible d'afficher l'intégralité de la communication sur une seule capture exploitable. L'analyse s'est donc concentrée volontairement sur le début de la trame, qui contient les éléments les plus significatifs pour la validation du protocole.

La capture réalisée met en évidence la condition de départ (START), caractérisée par une transition de la ligne SDA de l'état haut vers l'état bas alors que la ligne SCL est maintenue à l'état haut. Cette condition marque l'initialisation de la communication par le maître. Les impulsions régulières observées sur la ligne SCL confirment que l'horloge est correctement générée par le microcontrôleur.

Les premiers bits transmis sur la ligne SDA correspondent à l'adresse du capteur suivie du bit de lecture ou d'écriture. La synchronisation entre les données et l'horloge est conforme aux spécifications du protocole I²C, les changements d'état de la ligne SDA n'intervenant que lorsque la ligne SCL est à l'état bas. La présence d'un bit d'acquittement (ACK) après l'émission de l'adresse confirme que le capteur SHT30 répond correctement aux requêtes du maître.

L'utilisation d'un oscilloscope numérique permet de figer précisément ces instants clés de la communication, facilitant ainsi l'identification des différentes phases du protocole et la vérification du respect des contraintes temporelles imposées par le protocole I²C. Même si seule une portion de la trame a été analysée, cette observation est suffisante pour valider le bon déroulement de l'initialisation de la communication et la correcte reconnaissance du capteur sur le bus I²C.

3.8 Capteur BMP280 (Equipe3)

3.8.1 Présentation du BMP280



Image 13 - Capteur BMP280

Le BMP280 est un capteur barométrique absolu spécialement conçu par Bosch. Il est intégré dans un boîtier compact, ce qui, combiné à sa faible consommation d'énergie, permet son utilisation dans des appareils alimentés par batterie. Ce capteur repose sur la technologie piézo-résistive offrant une grande précision, linéarité, ainsi qu'une stabilité à long terme et une robustesse face aux perturbations électromagnétiques. Il est composé d'une membrane en silicium qui se déforme sous la pression, ce qui modifie sa résistance électrique, un changement converti en un signal de tension proportionnel à la pression. Il est utilisé dans diverses applications comme la mesure de mouvement dans des montres connectées et jouets, systèmes de navigation, détection d'élévation, station météo, ...

Le capteur a une gamme de mesure de 300 à 1100hPa (équivalent à +9000m jusqu'à -500m) avec une précision relative de $\pm 0.12\text{hPa}$ (équivalent à $\pm 1\text{m}$). Il peut communiquer via I²C (jusqu'à 3,4 MHz) ou SPI (jusqu'à 10 MHz). Sa consommation est très faible : $2,7\ \mu\text{A}$ à un taux d'échantillonnage de 1 Hz. Il fonctionne dans une plage de température allant de $-40\ ^\circ\text{C}$ à $+85\ ^\circ\text{C}$.



Image 14 - Capteur BMP280

Ce module à 3 modes de fonctionnement, le Sleep mode, le Normal mode et le Forced mode. La période de mesure du BMP280 comprend une mesure de température et de pression avec un suréchantillonnage sélectionnable. Après la période de mesure, les données sont traitées par un filtre

IIR (*Infinite Impulse Response*) optionnel qui élimine les fluctuations de pression de courte durée (par exemple, celles causées par un coup de vent). Nous avons utilisé une lecture en rafale (burst read) pour récupérer les données brutes, plutôt que de lire chaque registre séparément. Ensuite, les données sont introduites dans une formule de compensation pour obtenir les valeurs de pression et de température finales et normalisées.

3.8.2 Initialisation

Use case	Mode	Over-sampling setting	osrs_p	osrs_t	IIR filter coeff. (see 3.3.3)	Timing	ODR [Hz] (see 3.8.2)	BW [Hz] (see 3.3.3)
handheld device low-power (e.g. Android)	Normal	Ultra high resolution	×16	×2	4	t _{standby} = 62.5 ms	10.0	0.92
handheld device dynamic (e.g. Android)	Normal	Standard resolution	×4	×1	16	t _{standby} = 0.5 ms	83.3	1.75
Weather monitoring (lowest power)	Forced	Ultra low power	×1	×1	Off	1/min	1/60	full
Elevator / floor change detection	Normal	Standard resolution	×4	×1	4	t _{standby} = 125 ms	7.3	0.67
Drop detection	Normal	Low power	×2	×1	Off	t _{standby} = 0.5 ms	125	full
Indoor navigation	Normal	Ultra high resolution	×16	×2	16	t _{standby} = 0.5 ms	26.3	0.55

Tableau 3 - Tableau de configuration du capteur BMP280

Pour configurer le capteur BMP280, nous nous sommes référés à la documentation officielle fournie par Bosch. Celle-ci inclut un tableau récapitulant les réglages à appliquer au capteur en fonction des différents cas d'utilisation.

La configuration Weather Monitoring est pensée pour fournir des mesures atmosphériques fiables tout en maintenant une consommation énergétique faible. Elle ne cherche pas à maximiser la précision au niveau le plus élevé possible, mais à obtenir une résolution suffisante pour le suivi de tendances météorologiques, ce qui est son objectif principal.

Dans cette configuration, la pression et la température utilisent un oversampling faible (typiquement ×1). Ce niveau d'oversampling suffit pour obtenir une mesure stable et exploitable, tout en réduisant le temps de mesure et la consommation électrique. La température est mesurée principalement pour permettre la compensation interne de la pression.

Le capteur fonctionne en mode Forced, ce qui signifie qu'il ne réalise pas de mesures en continu. Chaque acquisition doit être déclenchée par le logiciel ; après la mesure, le capteur retourne automatiquement en mode veille. Ce mode est particulièrement adapté aux systèmes économes en énergie.

Enfin, un filtre interne léger peut être activé pour lisser les fluctuations dues au bruit. Cela stabilise la sortie sans introduire une latence importante, ce qui est suffisant pour du suivi météorologique où les variations rapides ne sont pas critiques.

3.8.3 Reste du code

Le fichier `bmp280.c` regroupe l'ensemble des fonctions nécessaires à l'initialisation et à l'exploitation du capteur de pression et de température BMP280 via l'interface I²C. Ce module assure non seulement la communication avec le capteur, mais également la reconstruction des données brutes et leur compensation conformément aux équations fournies par le constructeur Bosch dans la documentation technique.

La première fonction développée est `bmp280_init()`, qui configure le BMP280 en mode « weather monitoring ». Dans ce mode, le capteur fonctionne en forced mode, ce qui signifie qu'une seule mesure est effectuée à la demande, puis le capteur retourne automatiquement en veille.

L'initialisation active un suréchantillonnage $\times 1$ pour la température et $\times 1$ pour la pression, tout en désactivant le filtre interne. Les registres `CTRL_MEAS` et `CONFIG` sont écrits via le bus I²C, et la fonction retourne une valeur de type `app_err_t` indiquant la réussite de la configuration ou l'échec d'une communication I²C.

La seconde fonction importante est `bmp280_read_raw()`, dont le rôle est de déclencher une nouvelle mesure et de récupérer les valeurs brutes fournies par le capteur. Cette fonction commence par relancer une conversion grâce à `bmp280_init()`, puis surveille le registre `STATUS` du BMP280 afin de détecter la fin de la mesure. Une fois la conversion terminée, elle lit en mode « burst » six octets consécutifs contenant les données non compensées de température et de pression. Ces données sont ensuite recomposées en valeurs 20 bits conformément au format imposé par Bosch. Les valeurs brutes sont renvoyées via deux pointeurs passés en paramètre.

Pour pouvoir interpréter correctement les valeurs brutes, il est nécessaire d'appliquer les coefficients d'étalonnage internes propres à chaque capteur. La fonction `bmp280_read_calibration()` se charge de lire les 24 octets stockés dans l'EEPROM interne du BMP280. Ces coefficients, nommés T1 à T3 pour la température et P1 à P9 pour la pression, sont indispensables aux calculs de compensation. La fonction reconstruit chaque coefficient dans une structure dédiée, en respectant l'ordre et le format définis dans la datasheet.

La compensation de la température est assurée par la fonction `bmp280_compensate_T_int32()`, qui applique pas à pas les équations recommandées par Bosch. Elle reçoit la valeur brute de température ainsi que les coefficients T1, T2 et T3, et calcule la température réelle en centi-degrés Celsius. Cette fonction génère également la variable intermédiaire `t_fine`, utilisée ensuite pour la compensation de pression. Le résultat final permet d'obtenir une température précise sans recours à des opérations flottantes, ce qui est essentiel sur microcontrôleur 8 bits.

```
// Returns temperature in DegC, resolution is 0.01 DegC. Output value of "5123" equals 51.23 DegC.
// t_fine carries fine temperature as global value
BMP280_S32_t t_fine;
BMP280_S32_t bmp280_compensate_T_int32(BMP280_S32_t adc_T)
{
    BMP280_S32_t var1, var2, T;
    var1 = (((adc_T >> 3) - ((BMP280_S32_t)dig_T1 << 1))) * ((BMP280_S32_t)dig_T2) >> 11;
    var2 = (((((adc_T >> 4) - ((BMP280_S32_t)dig_T1)) * ((adc_T >> 4) - ((BMP280_S32_t)dig_T1))) >> 12) *
        ((BMP280_S32_t)dig_T3)) >> 14;
    t_fine = var1 + var2;
    T = (t_fine * 5 + 128) >> 8;
    return T;
}
```

Image 15 - Formule de compensation de la température de la documentation du BMP280

La compensation de la pression est réalisée par `bmp280_compensate_P_int32()`, qui utilise la valeur brute de pression, l'ensemble des coefficients P1 à P9 et la variable `t_fine`. Cette fonction met en œuvre une série de calculs entiers en 32 bits, comme imposé par la documentation officielle. Le résultat final correspond à la pression atmosphérique exprimée en Pascal. Les équations implémentées sont celles recommandées pour maximiser la précision tout en garantissant la compatibilité avec des microcontrôleurs disposant de ressources limitées.

```
// Returns pressure in Pa as unsigned 32 bit integer. Output value of "96386" equals 96386 Pa = 963.86 hPa
BMP280_U32_t bmp280_compensate_P_int32(BMP280_S32_t adc_P)
{
    BMP280_S32_t var1, var2;
    BMP280_U32_t p;
    var1 = (((BMP280_S32_t)t_fine) >> 1) - (BMP280_S32_t)64000;
    var2 = (((var1 >> 2) * (var1 >> 2)) >> 11) * ((BMP280_S32_t)dig_P6);
    var2 = var2 + ((var1 * ((BMP280_S32_t)dig_P5) << 1);
    var2 = (var2 >> 2) + (((BMP280_S32_t)dig_P4) << 16);
    var1 = (((dig_P3 * (((var1 >> 2) * (var1 >> 2)) >> 13)) >> 3) + (((BMP280_S32_t)dig_P2 * var1) >> 1)) >> 18;
    var1 = (((32768 + var1)) * ((BMP280_S32_t)dig_P1) >> 15);
    if (var1 == 0)
    {
        return 0; // avoid exception caused by division by zero
    }
    p = (((BMP280_U32_t)((BMP280_S32_t)1048576 - adc_P) - (var2 >> 12)) * 3125);
    if (p < 0x80000000)
    {
        p = (p << 1) / ((BMP280_U32_t)var1);
    }
    else
    {
        p = (p / (BMP280_U32_t)var1) * 2;
    }
    var1 = (((BMP280_S32_t)dig_P9) * ((BMP280_S32_t)((p >> 3) * (p >> 3)) >> 13)) >> 12;
    var2 = (((BMP280_S32_t)(p >> 2)) * ((BMP280_S32_t)dig_P8)) >> 13;
    p = (BMP280_U32_t)((BMP280_S32_t)p + ((var1 + var2 + dig_P7) >> 4));
    return p;
}
```

Image 16 - Formule de compensation de la pression de la documentation du BMP280

Enfin, la fonction principale du module est `bmp280_read()`, qui constitue l'interface de haut niveau utilisée par les autres parties du logiciel. Cette fonction coordonne l'ensemble des opérations nécessaires à une mesure complète : rappel de la fonction d'initialisation, lecture des valeurs brutes, récupération des coefficients d'étalonnage, compensation de la température, puis compensation de la pression. Les résultats sont ensuite placés dans une structure `bmp280_data_t`, contenant la température exprimée en centi-degrés Celsius et la pression exprimée en Pascal. Cette fonction permet donc d'obtenir en un seul appel toutes les données nécessaires au fonctionnement d'une station météo embarquée.

L'ensemble de ces fonctions rend le module BMP280 autonome, fiable et conforme aux exigences du constructeur. Il peut être intégré directement dans une application embarquée telle qu'un système de mesure environnementale, une station météo pédagogique ou un datalogger.

3.8.4 Analyse à l'oscilloscope

(I²C) Nous avons également analysé les signaux reçus par le capteur BMP280 grâce à un oscilloscope.

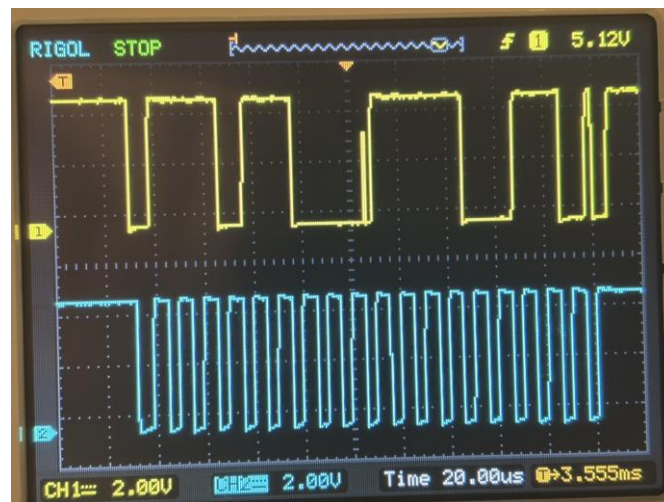


Image 17 - Octet reçu par le BMP280 capturé avec l'oscilloscope

Les signaux capturés sur l'oscilloscope sont difficilement identifiables, malgré cela on peut quand même reconnaître l'octet ('0111 0110') sur la **Erreur ! Source du renvoi introuvable.** qui correspond à la valeur 'EC', qui est l'adresse d'écriture du capteur BMP280. Le neuvième bit est l'acknowledge qui doit être à 0. L'octet suivant correspondrait à l'adresse du registre dans lequel écrire. Le deuxième octet que nous pouvons lire ('1111 0011') correspond à l'adresse F3, qui contient le statut actuel du capteur, cette communication sert donc probablement à vérifier le statut du capteur afin de savoir s'il a fini sa mesure. Ce qui permettra par la suite de récupérer les valeurs de température et de pression.

3.9 Equipe application (Equipe6)

3.9.1 Présentation du Bluetooth HC-05

Le HC-05 est un module Bluetooth permettant d'établir une communication sans fil entre deux appareils. Il s'agit d'un module très répandu dans le domaine des systèmes embarqués en raison de sa simplicité d'utilisation et de son faible coût. Ce module intègre une puce Bluetooth standard ainsi qu'une interface série UART, ce qui lui permet de fonctionner comme un pont transparent entre une liaison série filaire et une connexion Bluetooth.

Dans le cadre de notre projet de station météo, le HC-05 assure la transmission des données entre le microcontrôleur PIC18F26K83 et l'application mobile. Il permet ainsi à l'utilisateur de récupérer les mesures enregistrées par le datalogger sans avoir besoin d'une connexion physique avec la station.

3.9.2 Caractéristiques techniques

Le module HC-05 repose sur la norme Bluetooth 2.0 avec EDR (Enhanced Data Rate). Cette technologie, bien que plus ancienne que le Bluetooth Low Energy (BLE), offre l'avantage d'un débit de données plus élevé et d'une compatibilité native avec le profil SPP (Serial Port Profile), qui émule une liaison série classique.

Les principales caractéristiques du module sont les suivantes :

- Fréquence de fonctionnement : 2.4GHz (bande ISM)
- Portée : environ 10 mètres en environnement intérieur
- Tension d'alimentation : 3.3V à 6V (régulateur intégré)
- Consommation : 30 à 40 mA en communication, 8mA en veille
- Sensibilité : -80dBm
- Puissance d'émission : +4dBm (classe 2)
- Débit maximal : 2.1 Mbps (EDR), 721kpbs en pratique via SPP

Le module peut fonctionner selon deux modes : le mode esclave, qui est le mode par défaut où le module attend qu'un appareil maître initie la connexion, et le mode maître, où le module peut lui-même établir une connexion vers un autre périphérique Bluetooth. Dans notre application, le HC-05 est configuré en mode esclave : c'est le smartphone qui initie la connexion vers la station météo.

3.9.3 Interface UART

La communication entre le microcontrôleur et le module HC-05 s'effectue via une liaison série asynchrone UART (Universal Asynchronous Receiver-Transmitter). Cette interface nécessite uniquement deux fils de données : TX (transmission) et RX (réception), en plus de l'alimentation.

Les paramètres de communication par défaut sont :

Paramètre	Valeur
Baudrate	9600 bauds
Bits de données	8
Parité	Aucune
Bits de stop	1

Tableau 4 - Paramètre de communication pour l'UART

Ces paramètres peuvent être modifiés via les commandes AT lorsque le module est placé en mode configuration. Cependant, pour notre application, les paramètres par défaut sont suffisants et garantissent une communication fiable à faible vitesse, adaptée au volume de données à transmettre.

BROCHE	FONCTION	CONNEXION
VCC	Alimentation positive	+5V (ou 3.3V)
GND	Masse	GND du PIC
TXD	Sortie données série	Broche RX du PIC
RXD	Entrée données série	Broche TX du PIC
STATE	Indicateur de connexion	Non utilisée
EN/KEY	Activation mode AT	Non utilisée

Tableau 5 – Câblage du module HC-05

Il est important de noter le croisement des lignes TX et RX : la broche TX du HC-05 doit être reliée à la broche RX du microcontrôleur, et inversement. De plus, si le PIC fonctionne en logique 3.3V, la connexion peut être directe. En revanche, si des niveaux 5V sont utilisés, un diviseur de tension peut être nécessaire sur la ligne RX du HC-05 pour éviter d'endommager le module.

3.9.4 Principe de fonctionnement

Le fonctionnement du HC-05 repose sur le concept de liaison série transparente. Une fois la connexion Bluetooth établie entre le smartphone et le module, ce dernier agit comme un simple relais : tout octet reçu sur sa broche RX est transmis via Bluetooth au smartphone, et inversement, tout octet reçu via Bluetooth est retransmis sur sa broche TX vers le microcontrôleur.

Ce fonctionnement transparent présente un avantage considérable : il n'est pas nécessaire d'implémenter une pile protocolaire Bluetooth complexe sur le microcontrôleur. Du point de vue du firmware du PIC, la communication avec le smartphone est identique à une communication série filaire classique. Cela simplifie grandement le développement et le débogage.

3.9.5 Rôle dans le projet

Dans notre station météo, le module HC-05 remplit le rôle d'interface de communication entre le système embarqué et l'utilisateur. Il permet :

- La récupération des données : l'utilisateur peut synchroniser les mesures stockées dans le datalogger directement sur son smartphone, sans manipulation physique de la station.
- Une installation flexible : la communication sans fil permet de placer la station météo dans un endroit optimal pour les mesures (extérieur, hauteur) tout en conservant un accès facile aux données.

Le choix du HC-05 par rapport à d'autres solutions se justifie par plusieurs facteurs.

Premièrement, sa simplicité d'intégration : aucune pile Bluetooth à implémenter côté microcontrôleur. Deuxièmement, sa compatibilité avec les smartphones Android via le profil SPP. Troisièmement, son coût très faible (environ 5€), adapté à un projet de cette envergure.

Il convient toutefois de noter une limitation : le Bluetooth Classic n'est pas supporté par iOS pour les applications tierces. Apple impose l'utilisation du Bluetooth Low Energy (BLE) pour les accessoires non certifiés MFi. Notre application est donc limitée aux appareils Android.

Une évolution future du projet pourrait intégrer un module BLE (comme le HM-10) pour assurer la compatibilité iOS.

3.9.6 Principe de la communication Bluetooth

La communication entre l'application mobile et le microcontrôleur repose sur un protocole applicatif simple, conçu spécifiquement pour ce projet. Comme expliqué précédemment, le module HC-05 assure une liaison série transparente : du point de vue logiciel, l'échange de données s'effectue comme si le smartphone était directement connecté au PIC via un câble série.

Le protocole adopte une architecture de type commande-réponse. L'application mobile joue le rôle de maître : elle envoie des commandes au microcontrôleur, qui répond avec les données demandées. Cette approche garantit une synchronisation claire entre les deux parties et évite les conflits pouvant survenir si les deux côtés tentaient d'émettre simultanément.

Chaque échange suit le schéma suivant :

- L'application envoie une commande (un seul caractère ASCII)
- Le microcontrôleur traite la commande
- Le microcontrôleur renvoie la réponse appropriée
- L'application parse et exploite la réponse

3.9.7 Protocole applicatif

Le protocole définit quatre commandes permettant de récupérer l'ensemble des informations nécessaires à la synchronisation des données météorologiques. Chaque commande est constituée d'un unique caractère ASCII, ce qui simplifie le parsing côté firmware et réduit le risque d'erreurs de transmission.

Les commandes implémentées sont les suivantes :

COMMANDE	DESCRIPTION	FORMAT DE LA RÉPONSE
C	Demande de configuration	HHMM ;DDMM ;période\r\n
N	Nombre de records stockés	count\r\n
R	Réinitialisation de l'index de lecture	OK\r\n
A	Lecture du prochain enregistrement	idx;T;H;P\r\n

Tableau 6 – Liste des commandes disponibles

Toutes les réponses sont terminées par les caractères de retour chariot et saut de ligne (\r\n), ce qui permet à l'application de détecter facilement la fin d'une trame.

Commande C - Configuration

Cette commande permet de récupérer les paramètres de configuration du datalogger. La réponse contient trois informations séparées par des points-virgules :

- HHMM : heure et minute de début d'enregistrement (format 24h, sur 4 chiffres)
- DDMM : jour et mois de début d'enregistrement (sur 4 chiffres)
- Période : intervalle entre deux mesures, exprimé en minutes

Par exemple, la réponse 0800;2012;1 indique que l'enregistrement a débuté le 20 décembre à 08h00, avec une mesure toutes les minutes. Ces informations sont essentielles pour reconstituer l'horodatage de chaque mesure, comme nous le verrons dans la section suivante.

Commande N - Nombre de records

Cette commande retourne simplement le nombre total d'enregistrements présents dans la mémoire du datalogger. Cette valeur permet à l'application de connaître le volume de données à synchroniser et d'afficher une progression lors du téléchargement.

Commande R - Reset de l'index

Le datalogger maintient un index de lecture interne qui s'incrémente à chaque appel de la commande A. La commande R permet de réinitialiser cet index à zéro, afin de pouvoir relire l'ensemble des enregistrements depuis le début. La réponse « OK » confirme que l'opération a été effectuée avec succès.

Commande A - Lecture d'un enregistrement

Cette commande retourne l'enregistrement situé à l'index courant, puis incrémente automatiquement cet index. La réponse contient quatre valeurs séparées par des points-virgules :

- Idx : numéro de l'enregistrement (0 à n-1)
- T : température en centième de degrés Celsius
- H : humidité relative en centième de pourcent
- P : pression atmosphérique en Pascals

Le choix de transmettre les valeurs sous forme d'entiers plutôt que de nombres à virgule flottante s'explique par plusieurs raisons. D'une, cela simplifie le traitement côté microcontrôleur qui n'a pas à gérer le formatage des décimales. D'autre part, cela garantit une précision constante et évite les problèmes d'arrondi liés à la représentation binaire des flottants.

Par exemple, la réponse 42;2350;4550;101325 correspond à :

- Enregistrement n°42
- Température : 23.50 °C
- Humidité : 45.50%
- Pression : 1013.25 hPa

3.9.8 Liaison série UART

La communication physique entre le PIC et le module HC-05 s'effectue via le protocole UART. Il s'agit d'une transmission série asynchrone, ce qui signifie qu'aucune horloge commune n'est partagée entre l'émetteur et le récepteur. La synchronisation s'effectue grâce au bit de start qui précède chaque octet transmis.

Les paramètres de la liaison sont configurés comme suit :

- Baudrate : 9600 bits par secondes
- Format : 8 bits de données, pas de parité, 1 bit de stop
- Durée d'un bit : $1/9600 = 104.17 \mu s$

Chaque trame UART se compose donc de 10 bits au total : 1 bit de start (niveau bas), 8 bits de données (LSB en premier), et 1 bit de stop (niveau haut). Pour transmettre un caractère ASCII, il faut donc environ 1.04ms.

3.9.9 Oscillogrammes

Les mesures suivantes ont été réalisées à l'oscilloscope afin de valider le bon fonctionnement de la communication et de visualiser les trames échangées entre le microcontrôleur et le module Bluetooth.



Image 18 - Communication UART : envoi d'une commande

Cet oscillogramme montre la transmission du caractère 'A' depuis le module Bluetooth vers le smartphone.



Image 19 - Communication UART : Réception d'une commande

3.9.10 Programmation de l'app

Le développement de l'application mobile repose sur React Native, un framework open-source créé par Meta (anciennement Facebook) permettant de créer des applications natives pour Android et iOS à partir d'une base de code unique écrite en JavaScript ou TypeScript. Ce choix s'est imposé pour plusieurs raisons.

Premièrement, la portabilité du code. Plutôt que de développer deux applications distinctes (une en Java/Kotlin pour Android, une en Swift pour iOS), React Native permet d'écrire une seule fois la logique applicative et l'interface utilisateur. Bien que notre projet soit actuellement limité à Android en raison des restrictions d'Apple sur le Bluetooth Classic, cette architecture facilitera une éventuelle migration vers le Bluetooth Low Energy et la compatibilité iOS.

Deuxièmement, la productivité de développement. React Native offre des fonctionnalités comme le Hot Reload, qui permet de visualiser instantanément les modifications du code sans recompiler entièrement l'application. Cette caractéristique accélère considérablement les cycles de développement et de débogage.

Troisièmement, l'écosystème riche. La communauté React Native propose de nombreuses bibliothèques tierces couvrant la plupart des besoins courants, y compris la communication Bluetooth.

Le tableau suivant résume les principales technologies utilisées et leur rôle dans le projet :

TECHNOLOGIE	ROLE	JUSTIFICATION
REACT NATIVE	Framework de développement mobile	Application cross-platform avec une seule base de code
EXPO	Environnement de développement	Simplifie la configuration, le build et le déploiement
TYPESCRIPT	Langage de programmation	Typage statique réduisant les erreurs à l'exécution
REACT-NATIVE-BLUETOOTH-CLASSIC	Communication Bluetooth	Seule bibliothèque supportant le profil SPP sur React Native
ASYNC STORAGE	Stockage persistant	Sauvegarde locale des données et de la configuration
EXPO-FILE-SYSTEM	Gestion des fichiers	Création des fichiers d'export CSV et JSON
EXPO-SHARING	Partage de fichiers	Permet d'envoyer les exports par email, messagerie, etc.

Tableau 7 - Technologies proposées par React Native utilisées dans le projet

L'utilisation d'Expo mérite une explication particulière. Expo est une plateforme qui simplifie le développement React Native en fournissant un ensemble d'outils préconfigurés et une bibliothèque de composants prêts à l'emploi. Cependant, la bibliothèque Bluetooth que nous utilisons nécessite des modules natifs personnalisés, incompatibles avec l'environnement standard "Expo Go". Nous avons donc dû migrer vers un "Development Build", qui permet d'intégrer des modules natifs tout en conservant les avantages d'Expo.

3.9.11 Architecture Logicielle

L'application suit une architecture basée sur le pattern Context API de React. Ce pattern permet de gérer un état global partagé entre tous les composants de l'application, sans avoir à transmettre manuellement les données de composant en composant.

Le cœur de l'application réside dans un fichier central appelé « ConnectionContext », qui joue le rôle de gestionnaire global. Ce contexte centralise l'ensemble des données et des fonctions nécessaires au fonctionnement de l'application :

Données gérées par le contexte :

- L'état de la connexion Bluetooth (appareil connecté, type de connexion, statut)
- Les données météorologiques actuelles (température, humidité, pression, horodatage)
- L'historique complet des mesures synchronisées
- La configuration du datalogger (date de départ, période d'échantillonnage)
- Les indicateurs d'état (synchronisation en cours, chargement)

Fonctions exposées par le contexte :

- Scan et connexion aux appareils Bluetooth
- Envoi des commandes au microcontrôleur
- Synchronisation des données
- Export en CSV et JSON
- Gestion de l'historique local

Cette centralisation présente plusieurs avantages. Elle garantit une cohérence des données à travers toute l'application : lorsqu'une nouvelle mesure est reçue, tous les écrans affichant cette information sont automatiquement mis à jour. Elle facilite également la maintenance du code en regroupant toute la logique de communication en un seul endroit.

L'application se compose de quatre écrans principaux, chacun accédant au contexte partagé pour obtenir les données dont il a besoin :

- **Écran d'accueil** : point d'entrée de l'application, affichant le statut général
- **Écran Dashboard** : affichage des dernières mesures reçues
- **Écran Connexion** : liste des appareils Bluetooth disponibles et gestion de la connexion
- **Écran Historique** : consultation des mesures passées, statistiques et export des données

3.9.12 *Gestion de la connexion Bluetooth*

La connexion Bluetooth constitue le point d'entrée de toute interaction avec la station météo.

L'application utilise la bibliothèque react-native-bluetooth-classic qui implémente le profil SPP (Serial Port Profile), permettant d'émuler une liaison série via Bluetooth.

Scan des appareils disponibles

Avant d'établir une connexion, l'application doit identifier les appareils Bluetooth disponibles. Sur Android, cela se fait en récupérant la liste des appareils appairés, c'est-à-dire les appareils avec lesquels le téléphone a déjà établi une relation de confiance via les paramètres système.

Il est important de noter que l'utilisateur doit préalablement appairer manuellement le module HC-05 dans les paramètres Bluetooth de son téléphone avant de pouvoir l'utiliser avec l'application. Le code PIN par défaut du HC-05 est généralement "1234" ou "0000". Une fois cet appairage effectué, l'application peut détecter le module dans la liste des appareils disponibles.

Établissement de la connexion

Lorsque l'utilisateur sélectionne le module HC-05 dans la liste, l'application initie le processus de connexion. Ce processus comprend plusieurs étapes pour garantir une connexion fiable :

- **Vérification de l'état actuel** : si une connexion existe déjà avec l'appareil, elle est d'abord fermée proprement pour éviter tout conflit.
- **Connexion avec paramètres UART** : la connexion est établie en spécifiant les paramètres de communication, notamment le délimiteur de trame (retour chariot et saut de ligne) et l'encodage des caractères (UTF-8).

- **Mise en place de l'écoute** : un mécanisme d'écoute est configuré pour recevoir les données entrantes de manière asynchrone.
- **Récupération initiale de la configuration** : après un court délai de stabilisation, l'application envoie automatiquement les commandes C et N pour récupérer la configuration du datalogger et le nombre d'enregistrements disponibles.

3.9.13 Envoi des commandes

L'envoi de commandes vers le microcontrôleur s'effectue via une fonction centralisée qui transmet les caractères via la connexion Bluetooth établie. Cette fonction gère également la journalisation des échanges à des fins de débogage.

Un point crucial doit être souligné concernant le format des commandes : elles sont envoyées sans aucun caractère de fin de ligne. Contrairement à de nombreux protocoles série qui attendent un retour chariot ou un saut de ligne pour marquer la fin d'une commande, le firmware de notre microcontrôleur traite chaque caractère individuellement dès sa réception. L'envoi d'un caractère supplémentaire comme \n serait interprété comme une commande distincte et inconnue, générant une réponse d'erreur qui perturberait le flux de communication.

Les quatre commandes du protocole (C, N, R, A) sont exposées via des fonctions dédiées dans l'interface du contexte, permettant aux différents écrans de l'application de déclencher facilement les actions correspondantes.

3.9.14 Traitement des données reçues

Le traitement des données reçues constitue une partie essentielle de l'application. Chaque trame reçue du microcontrôleur doit être analysée pour déterminer son type et extraire les informations qu'elle contient.

Identification du type de réponse

La première étape du traitement consiste à identifier le type de la réponse reçue. Cette identification s'effectue par analyse du format de la trame à l'aide d'expressions régulières, une technique de reconnaissance de motifs textuels :

- **Configuration** : une trame composée de quatre chiffres, un point-virgule, quatre chiffres, un point-virgule et un nombre (exemple : 0800;2012;1)
- **Compteur** : une trame composée uniquement d'un nombre entre 1 et 5 chiffres, sans point-virgule
- **Enregistrement de données** : une trame composée de quatre nombres séparés par des points-virgules, dont les deuxième et troisième peuvent être négatifs (exemple : 42;2350;4550;101325)
- **Messages système** : les réponses OK et ERR sont identifiées et traitées séparément

Cette approche par reconnaissance de motifs est plus robuste qu'un simple découpage par séparateur, car elle valide également le format global de la trame avant d'en extraire les données.

Parsing de la configuration

Lorsqu'une trame de configuration est identifiée, l'application en extrait les composantes individuelles. La partie horaire (quatre premiers chiffres) est découpée en heures et minutes. La partie date (quatre chiffres suivants) est découpée en jour et mois. La période d'échantillonnage est directement convertie en valeur numérique.

Une étape de validation vérifie la cohérence des valeurs extraites : les heures doivent être comprises entre 0 et 23, les minutes entre 0 et 59, les jours entre 1 et 31, les mois entre 1 et 12, et la période entre 1 et 1440 minutes (soit 24 heures maximum). Cette validation est essentielle pour éviter de stocker des configurations corrompues qui produiraient des horodatages incorrects.

Parsing des enregistrements

Les enregistrements de données sont découpés en leurs quatre composantes : index, température, humidité et pression. Les valeurs brutes sont ensuite converties en grandeurs physiques exploitables :

- La température, exprimée en centièmes de degrés, est divisée par 100 pour obtenir des degrés Celsius
- L'humidité, exprimée en centièmes de pourcent, est divisée par 100 pour obtenir un pourcentage
- La pression, exprimée en Pascals, est divisée par 100 pour obtenir des hectopascals

Une fois ces conversions effectuées, l'horodatage de la mesure est calculé à partir de la configuration du datalogger, puis l'enregistrement complet est ajouté à l'historique local.

3.9.15 *Processus de synchronisation*

La synchronisation complète des données est le processus par lequel l'application récupère l'ensemble des mesures stockées dans le datalogger. Ce processus se déroule en plusieurs étapes séquentielles.

Etapas de la synchronisation

- **Récupération de la configuration** : l'application envoie la commande C et attend la réponse contenant les paramètres du datalogger. Un délai d'attente maximal de deux secondes est prévu pour éviter un blocage en cas de problème de communication.
- **Récupération du nombre d'enregistrements** : la commande N est envoyée pour connaître le volume de données à télécharger.
- **Analyse des données existantes** : l'application compare les index des enregistrements déjà présents en local avec le nombre total annoncé par le datalogger, afin de déterminer combien de nouvelles mesures doivent être récupérées.
- **Réinitialisation de l'index de lecture** : la commande R remet à zéro le pointeur de lecture du datalogger, permettant de parcourir les enregistrements depuis le début.
- **Téléchargement des enregistrements** : une boucle envoie successivement la commande A pour récupérer chaque enregistrement. Un délai est observé entre chaque commande pour laisser au microcontrôleur le temps de traiter la requête et de transmettre la réponse.

3.9.16 *Persistance des données*

Les données synchronisées sont stockées localement sur le téléphone afin de rester disponibles même après la fermeture de l'application. Cette persistance est assurée par « AsyncStorage », une solution de stockage clé-valeur intégrée à React Native.

Deux catégories de données sont sauvegardées de manière persistante :

- **L'historique des mesures** : l'ensemble des enregistrements synchronisés, comprenant pour chacun l'index, l'horodatage calculé, les trois valeurs de mesure et la date de réception
- **La configuration du datalogger** : les paramètres de départ (date, heure) et la période d'échantillonnage

La sauvegarde de l'historique s'effectue automatiquement à chaque modification, garantissant qu'aucune donnée n'est perdue en cas de fermeture inattendue de l'application. Au démarrage, les données sauvegardées sont rechargées en mémoire, permettant à l'utilisateur de consulter immédiatement son historique sans avoir besoin de se reconnecter à la station météo.

4 Analyse du produit final

Lors de la dernière séance et après quelques ajustements de dernière minute nous avons pu arriver à une version finale satisfaisant le cahier des charges:

- Un menu navigué à partir des boutons pressoirs naviguant entre plusieurs sous-menus:
 - Configuration date et heure
 - Mesures de capteurs en temps réel
 - Configuration du laps de temps séparant les prises de mesures
 - Start/stop du datalogger
- Une application Android (et son image IOS) capable de visualiser la dernière mesure ainsi que de récupérer l'ensemble de l'historique de la même prise de mesures

Une piste d'amélioration serait l'emploi de la RTC pour une interruption à intervalle de temps régulier plus précise que celle utilisée provenant du microcontrôleur lui-même.

5 Conclusion

Pour conclure, ce projet peut être considéré comme une réussite, tant sur le plan des objectifs atteints que sur celui des compétences acquises. Même si l'utilité concrète du travail réalisé peut faire débat et ne pas faire l'unanimité, l'expérience a été particulièrement enrichissante. Le travail en groupe s'est déroulé dans un climat très positif : chacun a fait preuve de coopération, d'implication et d'écoute, ce qui a grandement facilité l'avancement du projet. Cette collaboration efficace a permis à l'ensemble du groupe d'apprendre et de progresser, faisant de cette expérience un point fort du projet.

6 Bibliographie

- Bosch Sensortec GmbH, BMP280 Digital Pressure Sensor Datasheet (BST-BMP280-DS001), Bosch Sensortec. <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf>
- Analog Devices Inc. / Maxim Integrated, DS1307 64 x 8, Serial, I²C Real-Time Clock Datasheet, Analog Devices product page. <https://www.mouser.be/datasheet/2/609/DS1307-3115548.pdf>
- iTeard Studio. (2010, 18 Juin). HC-05 Bluetooth Module Datasheet. Components101. https://components101.com/sites/default/files/component_datasheet/HC-05%20Datasheet.pdf
- Expo. (s.d.) Expo Documentation. <https://docs.expo.dev/>
- Davidsson, K. (s.d.). react-native-bluetooth-classic. GitHub. <https://github.com/kenj davidson/react-native-bluetooth-classic>
- Meta Platforms. (s.d.). React Native documentation. <https://reactnative.dev/docs/getting-started>
- React Native Community. (s.d.). @react-native-async-storage/async-storage. GitHub. <https://github.com/react-native-async-storage/async-storage>
- Chatterjee, S. (2022, 12 avril). Les 4 différents types d'horloges en temps réel et leur fonctionnement. Zipschedules Blog. <https://zipschedules.com/fr/blog/the-4-different-types-of-real-time-clocks-and-how-they-work.html>
- https://sensirion.com/media/documents/213E6A3B/63A5A569/Datasheet_SHT3x_DIS.pdf

7 Annexe

Weather Station – Rapport Architecture Logicielle – Victor Hachard -

<https://github.com/VictorHachard/applied-electronics-project-heh/blob/main/Weather%20Station%20-%20Rapport%20Architecture%20Logicielle.pdf>